META-LABELING ALPHA FILTER: A SYSTEMATIC SIGNAL REFINEMENT FRAMEWORK

A trading system that learns when not to trade

"All models are wrong, but some are useful." — George E. P. Box

by

Gautier Petit

github.com/gautierpetit
linkedin.com/in/gautierpetitch

August, 2025

Copyright (c) 2025 Gautier Petit
For research/education only. Not investment advice.
This document is licensed under the Creative Commons
Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).

Table of Contents

I — Executive Summary: 5
II — Motivation: 6
II.1 — The problem: signal quality, not direction 6
II.2 — The approach: decide *whether* to act, not just *how* 6
II.3 — Why meta-labeling instead of "a better alpha" 6
II.4 — What this build demonstrates 6
II.5 — Research objectives and success criteria 7
II.6 — Scope and boundaries 7
II.7 — Takeaway 7
III — Pipeline Overview 8
III.1 — From point-in-time data to learnable events 8
III.2 — Modeling, calibration, stacking, and execution 8
IV — Data & PIT Construction 10
IV.1 — Universe, dates, and point-in-time mapping 10
IV.2 — Data hygiene: de-dupe, near-zero guards, and spike neutralization 10
IV.3 — Saved artifacts and loaders (alignment by construction) 10
IV.4 — Sanity checks and reproducibility 11
IV.5 — Why point-in-time membership matters (bias audit) 11
IV.6 — Data gaps, failed downloads, and the coverage footprint 12
V. — Labeling Choices 13
V.1 — Objective and mechanics 13
V.2 — The design space: how PT, SL, and $tmax$ shape supervision 13
V.3 — Horizon choice at fixed thresholds: the (5σ, 5σ) sweep 14
V.4 — Final specification and its implications 15
V.5 — Practical considerations and robustness 16
VI — Modeling Decisions 17
VI.1 — Why these learners and what each is meant to learn 17
VI.2 — Training protocol and cross-validation 17
VI.3 — From scores to usable probabilities: calibration 17
VI.4 — The stacking model 18
VI.5 — From probabilities to trades 18

VI.6 — Fold-3 ablation (who wins, and by how much) 18
VI.7 — Calibration quality 20
VII — Execution & Portfolio Construction 21
VII.1 — Timing and return alignment 21
VII.2 — From probabilities to target weights 21
VII.3 — Churn controls and pathwise execution 21
VII.4 — Vol targeting and leverage cap 22
VII.5 — Transaction costs and executed PnL 22
VII.6 — Implementation notes and fixes 22
VIII — Evaluation Strategy 23
VIII.1 — Out-of-sample protocol and leakage control 23
VIII.2 — Portfolio-level metrics 23
VIII.3 — Calibration and classification diagnostics 23
VIII.4 — Benchmarks and portfolio variants 23
VIII.5 — Rebalancing, frictions, and realism 24
VIII.6 — Reporting artifacts and reproducibility 24
IX — Key Results 25
IX.1 — Performance at a glance (net of costs) 25
IX.2 — Diversification and beta profile 26
IX.4 — Rolling behavior and portfolio fit 27
IX.5 — Execution realism: turnover and leverage 28
IX.6 — Interpretation 28
X — Implementation Review: 29
X.1 — Challenges & Fixes 29
X.1.1 — Data & Reproducibility 29
X.1.2 — Labeling & Evaluation 29
X.1.3 — Execution & Portfolio Construction 30
X.1.4 — Practical Constraints 30
X.2 — Limitations 31
X.2.1 — Data fidelity and universe coverage 31
X.2.2 — Label design and horizon coupling 31
X.2.3 — Modeling and calibration caveats 31

X.2.4 — Execution realism, capacity, and costs 31
X.3 — Extensions 32
X.3.1 — Higher-fidelity data 32
X.3.2 — Side-aware modeling 32
X.3.3 — Adaptive acceptance 32
X.3.4 — Probability → size mappings (margin is the default) 32
X.3.5 — Barrier σ alternatives (labels only) 32
X.3.6 — Fundamentals sleeve (value or quality) 32
X.3.7 — Microstructure sleeve (short-horizon) 33
X.3.8 — Execution realism 33
X.3.9 — Learning-to-rank / listwise selection 33
X.3.10 — Conformal acceptance 33
X.3.11 — Policy learning for sizing 33
XI — Conclusion 34
XII — Appendix 35
III – Pipeline Overview 35
VI – Modeling Decisions 37
IX – Key results 43
Meta-Labeling Strategy 43
50/50 Blend with SPY 44
Beta Neutral 45

I — Executive Summary:

We build a meta-labeling filter that learns when a baseline cross-sectional momentum signal is likely to work and turns that insight into disciplined trade selection and sizing. Instead of "always trade the signal," our system predicts the probability that a proposed trade will succeed and acts only when conviction is high. The filter is engineered for deployment: probabilities are explicitly calibrated, trades are gated and sized off those probabilities, and execution hygiene (close → next-close accounting, executed-book costs, portfolio-level risk controls) is baked into the backtest so reported performance reflects how the sleeve would actually trade.

Out-of-sample, the sleeve stands on its own and pairs well with a core equity allocation. The pure meta-labeled strategy delivers 12.17% annualized return at 19.51% vol (Sharpe 0.62) with low, slightly negative equity exposure ($\beta \approx -0.11$, $\rho \approx -0.12$ to SPY). A simple 50/50 blend with SPY lifts risk-adjusted performance to 14.64% at 13.43% vol (Sharpe 1.09) and contains drawdowns (max −28.07% vs SPY −33.72%). A beta-neutral construction achieves 14.20% at 19.36% vol (Sharpe 0.73) with $\beta \approx 0$. Trade-level diagnostics confirm the filter's selectivity: 3,612 trades with a 65.75% win rate, profit factor 2.92, and an average holding period of 24.2 days. Full metrics in Section IX (Table IX.1)

The build emphasizes rigor over flash. Data and features are point-in-time; models are trained with chronology-respecting cross-validation and then calibrated; a classwise convex blender combines model views into a single, well-behaved probability used for gating and sizing; and costs, turnover, and volatility targeting are applied to the executed path, not targets. Every run writes manifests and config snapshots for reproducibility. The result is a deployable filter that upgrades a standard momentum sleeve into a probability-aware allocator and improves portfolio-level efficiency when blended with SPY.

II — Motivation:

II.1 — The problem: signal quality, not direction

Most alphas are noisy detectors of potential edge, not reliable trade directives. Even robust baselines like cross-sectional momentum cycle through high- and low-hit-rate regimes, suffer from crowding and volatility spikes, and leak PnL by acting on marginal setups. Tweaking thresholds, widening stops, or brute-force diversification rarely fixes the core error: we trade too often when the conditional odds are poor.

II.2 — The approach: decide *whether* to act, not just *how*

We predict the quality of each candidate trade before execution. Rather than asking "long or short?", we ask "should we act on this signal now, over a quarter-length horizon?" A meta-model estimates the probability that a proposed trade will resolve as a success (+1) rather than a failure (−1) or a timeout (0). Those probabilities drive decisions: we gate entries with calibrated thresholds and a margin over competing outcomes, and, when calibration is strong, map success probability to position size.

II.3 — Why meta-labeling instead of "a better alpha"

Meta-labeling upgrades any base strategy (such as: momentum, stat-arb, breakout) without re-deriving its alpha. It adds probabilistic control on top of existing entry logic: calibrated multiclass probabilities can be thresholded for selection and translated into sizing, turning a binary rule into a quality-aware process. Because the features include volatility state, market-stress proxies, and cross-sectional structure, the filter becomes regime-sensitive and learns when the environment actually allows the base edge to express. Practically, that means fewer but better trades, lower turnover and slippage, and a calmer drawdown profile that survives beyond toy backtests.

II.4 — What this build demonstrates

The project proceeds in a straight line: point-in-time data assembly flows into triple-barrier labeling, then feature construction at decision time; we fit the base models (LightGBM and MLP), calibrate their outputs, blend the calibrated distributions with a classwise convex mixer, pass the resulting success probabilities through a signal filter (threshold + margin + Top-K), map accepted signals to sizes, apply portfolio overlays (volatility targeting, leverage cap, and costs), and finally run an out-of-sample backtest. At each step chronology is enforced and leakage is policed, features are measured strictly at the decision date, cross-validation is time-aware, and only calibrated probabilities reach the filter and sizer.

We separate statistical validation from portfolio results. Model fitting and calibration are judged with log-loss and ROC-AUC alongside reliability diagnostics (per-class curves, ECE, Brier).

Backtest performance is then evaluated with risk-adjusted returns (Sharpe, Sortino), alpha–beta and SPY correlation, drawdowns and recovery, turnover, and capacity-aware costs.

II.5 — Research objectives and success criteria

We test whether a meta-filter can raise risk-adjusted returns by participating only when success probability is high; whether calibrated probabilities materially improve thresholding and sizing versus uncalibrated scores; and how the choice of PT/SL multipliers shifts class balance, coverage, and label noise. We also ask which regime signals (e.g., market volatility or cross-sectional crowding) best predict when the base edge works, and how much paper alpha survives after realistic turnover and asymmetric costs.

Success is defined by higher win rate and profit factor on traded events, better Sharpe/Sortino with equal or lower realized volatility, a shallower/faster-recovering drawdown profile, and improved reliability, lower calibration error and stable behavior across folds and time.

II.6 — Scope and boundaries

We work in long/short U.S. equities at daily frequency, with a cross-sectional momentum base, triple-barrier labels, and a meta-model that gates and sizes trades under realistic costs. We do not model intraday execution or alternative data, and we are not searching for a new alpha here, the point is to filter an existing one and turn it into a disciplined, probability-aware trading process.

II.7 — Takeaway

This project turns a decent signal into a professional decision process, one that knows when not to trade, allocates when the odds are favorable, and evaluates itself with the same probabilistic tools a live desk would use.

III — Pipeline Overview

III.1 — From point-in-time data to learnable events

The pipeline runs end-to-end from point-in-time (PIT) data assembly through labeling and feature construction, then into modeling and backtesting, with a single orchestrator (main.py) enforcing chronology. We operate with three non-overlapping folds: Fold-1 for model discovery, Fold-2 for out-of-sample calibration, and blender fitting, and Fold-3 for true out-of-sample trading and performance evaluation. Data ingestion applies PIT constituent masks and survivorship checks, augments the universe with SPY/VIX context, and writes OHLCV to parquet; loaders align frequencies and resample to monthly where the base signal requires it. The base strategy is a cross-sectional momentum allocator: it ranks on a monthly cadence and emits daily long/short participation signals, which also provide a natural "baseline momentum" comparator for evaluation. Each trade attempt is labeled with a triple-barrier scheme at a 63-day horizon: profit-taking and stop-loss thresholds are set at 5σ each, with $\sigma = \sigma_{63}$ measured as the 63-day rolling standard deviation of close-to-close returns; outcomes are determined on daily closes and recorded as successful (+1), timeout (0), or bad (−1). Features are computed strictly at decision time and combine short-horizon price structure (momentum term structure, volatility state, cross-sectional ranks and selected technicals) with regime context (market volatility level and term structure, simple macro proxies). We then de-duplicate and prune highly collinear columns, standardize where appropriate, and align X and Y with leakage-safe indexing so that every model only sees information available at its decision point.

III.2 — Modeling, calibration, stacking, and execution

Modeling proceeds in two layers. In Layer-1 we train a multiclass LightGBM with time-series cross-validation that includes a chronological gap, selected via a compact Bayesian search and refit with early stopping and class-balanced weights (GPU acceleration is used when available). In parallel, we train an MLP with nested time-series cross-validation; its training pipeline balances classes on the fly and uses batch normalization and dropout for stability.

Because decisions are probability-driven, both models are calibrated via a vector-scaled softmax: we learn per-class affine transforms of the logits on a rolling, leakage-free Fold-2 procedure and then carry the most recent calibrator into Fold-3. We evaluate the base models with macro-averaged ROC-AUC and log-loss, but also with calibration diagnostics (per-class reliability curves, Expected Calibration Error, and Brier scores) so that thresholds and sizes are grounded in well-behaved probabilities.

We prototyped an MLP stacker (available in an experimental branch) but ultimately adopted a classwise convex blender because it delivered the best out-of-sample calibration and log-loss as a stacker. The blender, fit on Fold-2, learns one weight per class to combine the calibrated LightGBM and MLP distributions; weights are constrained to (0,1) via a sigmoid and lightly shrunk toward 0.5. At inference, we reuse the learned weights with the Fold-3 calibrated base

models. The blender's success probability then gates trades with a hard threshold and a margin over the larger of the timeout and bad classes; when calibration is strong, we map that probability to size, normalize weights daily, add optional volatility targeting and a leverage cap, and apply asymmetric long/short costs. Finally, Fold-3 runs the out-of-sample backtest and reports cumulative and net PnL versus SPY and a standard momentum benchmark, turnover and leverage, rolling Sharpe and correlation, drawdowns, alpha–beta, and trade-level diagnostics such as hit rates, profit factor, and holding-time distributions. Configuration of fold dates, thresholds, costs, and search spaces is centralized in config.py, so the entire sequence is reproducible and auditable.

IV — Data & PIT Construction

IV.1 — Universe, dates, and point-in-time mapping

We build a U.S. large-cap equity universe from a historical S&P 500 constituents file and normalize symbols for Yahoo Finance compatibility. Ticker strings are sanitized with a dot → hyphen rule (so "BRK.B" becomes "BRK-B"), and only safe corporate renames are applied across the full history (e.g., FB → META, FISV → FI), avoiding aggressive remaps that could create spurious continuity. The constituent sets are expanded into a point-in-time mask: for each change date we flag only the tickers actually in the index from that date until the next event, intersecting with available columns so that any missing outside a name's true membership window is treated as out of universe rather than data loss. A final row is inserted at the end date so the mask covers the full requested range.

We then download daily OHLCV from Yahoo Finance (auto-adjusted) for the configured window, together with SPY and VIX as context. After PIT masking, we save the filtered closes/highs/lows/volumes to parquet for deterministic downstream use. Shapes and indexes are asserted to match between prices and the PIT mask, and we write a missing-data report and a ticker-availability report (percentage of valid days within the PIT window) for inspection.

IV.2 — Data hygiene: de-dupe, near-zero guards, and spike neutralization

Because the price feed comes from a free Yahoo Finance interface, which is unevenly maintained and prone to symbol quirks and sporadic glitches, we enforce explicit hygiene before anything touches the PIT mask. Before masking, duplicate ticker columns are removed by averaging duplicates to catch vendor quirks after normalization. Non-sensical prices (≤ 0 or effectively zero) are nulled to prevent exploding returns on pct_change. We neutralize absurd daily moves by setting to NaN any return with absolute magnitude above +500% (a conservative clip for feed glitches). We null anomalies rather than winsorize so downstream modules can choose whether to ignore or explicitly handle gaps, avoiding the hidden bias that comes from silently altering values.

IV.3 — Saved artifacts and loaders (alignment by construction)

All PIT-filtered artifacts (prices, high/low, volumes, SPY, and VIX) are saved to disk and read back through a small set of loaders that enforce consistent indexing. load_prices() returns the PIT-filtered close matrix; load_monthly_prices() offers calendar month-end resampling for the base momentum ranks; load_returns() computes simple daily returns off the filtered prices. SPY and VIX are stored as series and, in the loader, VIX is reindexed and forward-filled to the trading calendar so it aligns to equities. Optional macro rates (10Y and the 10Y–3M spread) are manually downloaded from FRED[1] then read from CSV, reindexed to the equity calendar, and

[1] Federal Reserve Bank of St. Louis:
- DGS10: https://fred.stlouisfed.org/series/DGS10

forward-filled in the same fashion. These loaders are intentionally minimal and fail fast if a configured file is missing, which keeps downstream modules clean and makes index mistakes obvious.

IV.4 — Sanity checks and reproducibility

Two categories of checks gate the dataset. Structural checks verify that the PIT mask shares index and columns with the raw price matrix before filtering, preventing silent misalignment. Content checks flag abnormal returns and produce human-readable reports, the spike neutralization logs top offenders, the missing-data and availability reports quantify any survivorship skew left after masking. The orchestrator saves a run manifest containing: Git SHA, Python/TensorFlow versions, random_state, label spec (PT, SL, t_{max}), fold dates, feature count and an MD5 of the feature columns, per-fold class priors, base-model hyperparameters, and the calibration/blender config; so results can be traced back to exact inputs. Additionally, class labels are stored in the manifest with the mapping $-1 \to 0, 0 \to 1, +1 \to 2$, which is the order used by the models and reports.

Alongside the manifest, we persist a config_snapshot.json that captures every runtime parameter and file path used in the run: data window, fold dates, triple-barrier settings (PT_SL_FACTOR = [5,5], MAX_HOLDING_PERIOD = 63, VOL_SPAN = 63), CV scheme (CV_N_SPLITS = 3, CV_GAP = 63), execution/cost settings (TARGET_VOL = 0.2, LEVERAGE_CAP = 3, LONG_SIDE_TC = 10 bps, SHORT_SIDE_TC = 20 bps), and filtering knobs (MIN_GAP = 0.10, META_PROBA_THRESHOLD_{LONG,SHORT} = {0.45,0.5}), as well as the exact hyperparameter search spaces. This makes runs reproducible and diffs auditable.

We also write fold_fingerprints.json: the length, head/tail keys, and a BLAKE2b sample hash for each fold; so any change to slicing or ordering is immediately detectable.

IV.5 — Why point-in-time membership matters (bias audit)

Using today's S&P 500 list to backtest the past bakes in look-ahead and survivorship bias: you implicitly "knew" which companies would endure and you ignore delisted underperformers. A third issue, namely selection bias, creeps in when you keep only long, clean histories, which selects for data-rich survivors. These biases inflate momentum backtests and degrade meta-model training because the model learns from information that did not exist at decision time. A point-in-time S&P 500 universe eliminates those assumptions: at each date we trade only names that were truly constituents then. That produces a more faithful picture of deployable performance and, crucially, higher-quality supervision for the meta-model because labels and features are conditioned on the actual investable set.

- T10Y3M: https://fred.stlouisfed.org/series/T10Y3M

IV.6 — Data gaps, failed downloads, and the coverage footprint

Over the 10-year window we successfully downloaded daily prices for the majority of PIT constituents, but 144 tickers failed due to delistings, symbol changes not covered by the "safe" map, or Yahoo time-zone/price unavailability errors. For each name we record missing-day counts and availability; examples include many historical components with 0% availability across our window (e.g., DISCK, HFC, TWC, CELG), which appear in missing_count.xlsx and ticker_availability.xlsx. Across the sample, the average ticker availability is ~76% within each name's PIT window. Our policy is straightforward and conservative: names with zero availability are excluded from trading and treated as out-of-universe for the entire window; names with partial availability participate only on days where both PIT membership and data coverage intersect. This avoids fabricating price paths while preserving the integrity of cross-sectional ranks.

We also note that excluding zero-availability components reduces historical breadth on certain dates. In practice, breadth remains sufficient for a cross-sectional momentum allocator; and because inclusion is governed by the PIT mask and actual data coverage, we avoid re-introducing survivorship via implicit "data completeness" selection. Finally, the constituents file is provided by Farrell J. Aultman under MIT License, distributed on GitHub[2]; manually downloaded at project start and snapshotted to disk.

In a nutshell, the PIT mask makes the investable set truly time-aware; hygiene and spike guards prevent obvious feed errors from leaking into labels or features; and the coverage policy treats missing data conservatively without re-introducing survival bias. The result is a self-documenting dataset that the rest of the pipeline can trust.

[2] The file S&P 500 Historical Components & Changes.csv is available on: https://github.com/fja05680/sp500/tree/master

V. — Labeling Choices

V.1 — Objective and mechanics

Our base strategy emits dated trade attempts. Each attempt is mapped to a categorical outcome using a triple-barrier scheme with three elements: a profit-taking threshold (PT), a stop-loss threshold (SL), and a vertical barrier that caps the maximum holding period t_{max}. PT and SL are not labels; they are thresholds used to judge the outcome. The labels are the outcomes themselves: successful (+1) if the profit threshold is touched first, bad (−1) if the stop threshold is touched first, and timeout (0) if neither threshold is reached before the horizon. Outcomes are evaluated on daily closes rather than intraday extremes. For a long position entered at close p_0, the thresholds are $p_0(1 + 5\sigma_{63})$ and $p_0(1 - 5\sigma_{63})$; for a short, the signs are reversed. We scale both thresholds by the per asset 63-day rolling standard deviation of close-to-close returns, σ_{63}, so the difficulty of achieving PT or hitting SL adapts to the prevailing noise level of each asset and regime. (Events with missing or zero σ_{63} at entry are skipped.)

This labeling decision fixes the learning problem for the meta-model. We are not trying to predict future returns directly, nor are we asking whether price will go up or down. We are asking a more operational question: *given that the base strategy proposes a trade today, what is the probability that this trade will be resolved as a success within one rebalancing quarter?* That choice of Y also constrains X. A short horizon would push us toward microstructure and very short-term momentum features; a quarter-length horizon naturally emphasizes momentum term structure, volatility state, and regime context. The label therefore acts as a contract: it specifies the timeframe and the mechanics the model must learn.

V.2 — The design space: how PT, SL, and t_{max} shape supervision

The three knobs PT, SL, and t_{max}, jointly control coverage, noise, and class balance. By coverage we mean the share of events that resolve as either +1 or −1 (i.e., $1 - Pr[timeout]$). Noise is more subtle: thresholds set too close to the entry price will be crossed by routine day-to-day volatility, producing many resolved labels that are not truly informative about setup quality; thresholds set too far will rarely be reached in time, starving the model of positive supervision and inflating timeouts. Class balance matters because extreme imbalance degrades both classification and probability calibration. We track it with simple proportions and with entropy $-\sum_y p_y \log p_y$ as a proxy for learnability: low entropy is a symptom of degeneracy, where one class dominates and the task becomes trivial. With those definitions in hand, it is

easier to read the PT/SL grid: moving the stop or the target changes not only the class proportions but the informativeness of the labels the model sees.

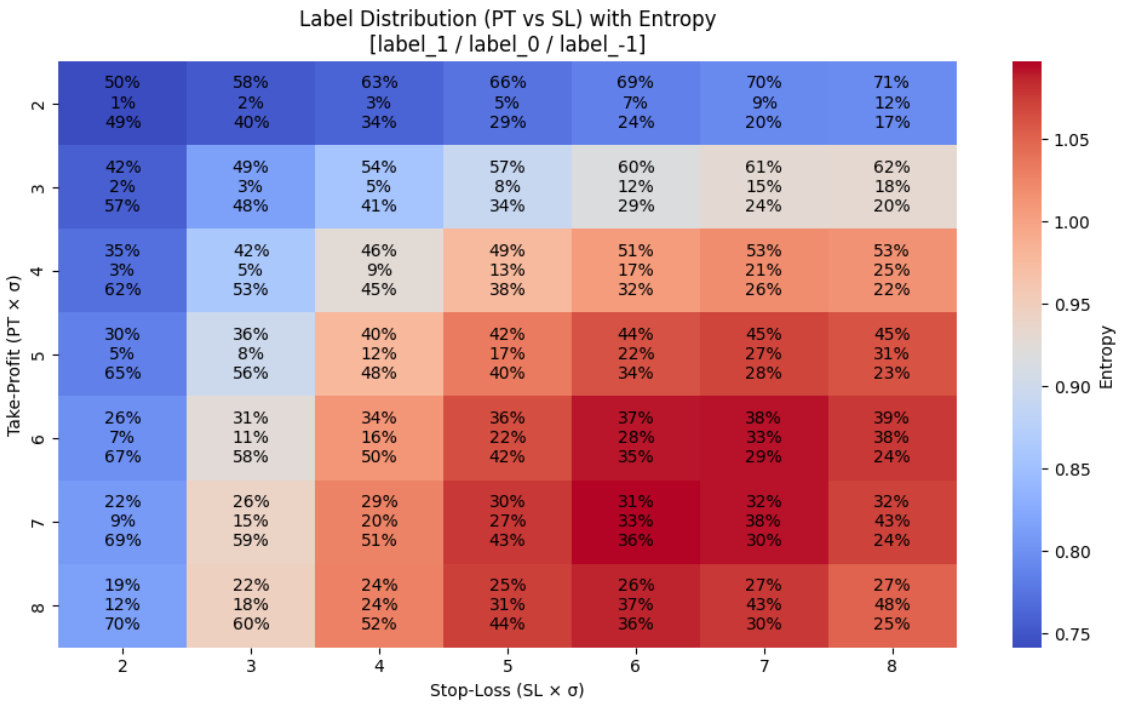

Figure V.1 — PT/SL grid at t_max=63 days (σ=63d). Cells show [successful/timeout/bad] and an entropy overlay; (5σ,5σ) lies in the high-entropy corridor with a non-trivial timeout class.

At a fixed horizon $t_{max} = 63$ days, sweeping PT and SL across a grid produces the expected geometry. Increasing the stop distance while holding PT fixed makes it harder to be stopped out and therefore shifts mass toward successful outcomes. Increasing the profit distance while holding SL fixed has the opposite effect, shifting mass toward bad outcomes. Moving both thresholds outward simultaneously increases the probability that neither is reached in time, raising timeouts and reducing coverage. The middle of the grid is where all three classes are represented in healthy proportions and where entropy is highest. That is exactly the region we want to train in: the model sees enough successes and failures to learn discriminative structure, and enough timeouts to learn when a setup is simply not likely to resolve within the horizon.

V.3 — Horizon choice at fixed thresholds: the (5σ, 5σ) sweep

Fixing PT and SL at (5σ, 5σ) and sweeping the maximum holding period clarifies how the horizon alone reshapes supervision. At very short horizons (e.g., 21 trading days) timeouts dominate because most moves of size $5\sigma_{63}$ do not complete that quickly. As we extend the horizon, timeouts decline monotonically and the mass they release is split between successful and bad outcomes: by 63 days, the shares stabilize around 46% successful, 12% timeout, and 43% bad. Pushing the horizon much beyond a quarter continues to reduce timeouts but yields diminishing informational returns, because the task drifts toward a near-binary competition between success and failure and begins to reflect slower dynamics than the base strategy targets.

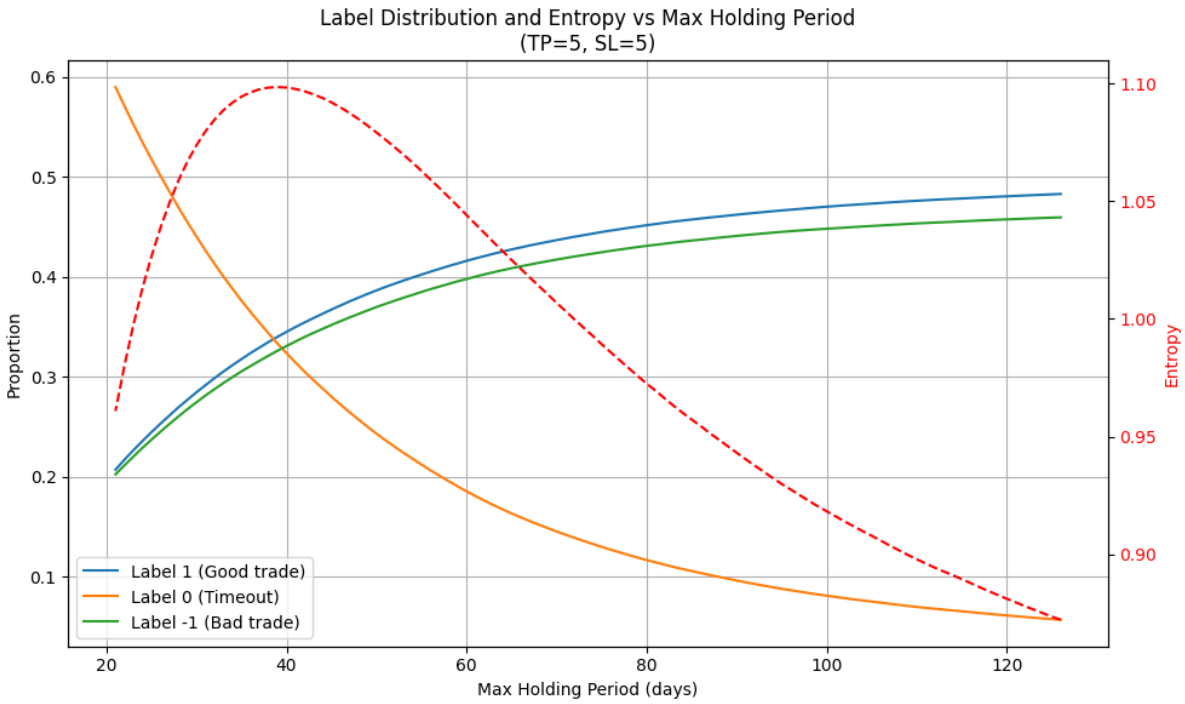

Figure V.2 — Label shares vs t_max at (5σ,5σ). Timeouts fall from ~0.59 (21d) to ~0.12 (63d); successful/bad rise to ~0.46/~0.43 at 63d and plateau thereafter.

This behavior explains our choice of $t_{max} = 63$. It aligns the label horizon with the base strategy's rebalancing cadence, keeps coverage high (approximately 88% at 63 days), and preserves a non-trivial timeout class that the meta-model can explicitly learn to avoid. In other words, the horizon is long enough to let genuine moves express, but not so long that we conflate multiple regimes or teach the model to wait indefinitely.

V.4 — Final specification and its implications

Throughout the report we adopt PT = 5σ, SL = 5σ, and $t_{max} = 63$ trading days, with σ measured as σ_{63}. This specification sits in the high-entropy corridor of the PT/SL grid and produces a label mix that is both informative and stable across time. Because the resolved outcomes are near-balanced at the chosen horizon, the model does not rely on heavy class re-weighting, which improves calibration. The presence of a clear timeout class is also valuable: it allows us to gate trades not only on the probability of success but also on whether the model expects the setup to resolve at all within the quarter.

Decision-making follows directly from these properties. We train a three-class classifier and use its calibrated probability of successful outcome as the primary control variable. Trades are admitted when this probability exceeds a threshold, and we optionally require a margin over the larger of the timeout and bad probabilities to eliminate ambiguous or slow setups. When calibration is strong, we map the success probability into position size so that conviction scales exposure rather than acting as a brittle on/off switch. This is exactly the kind of probabilistic control that a meta-filter should provide.

V.5 — Practical considerations and robustness

Two implementation details deserve emphasis because they influence interpretation. First, outcomes are determined using daily closes. This is conservative and reproducible: we do not assume knowledge of intraday highs or lows that we may not be able to execute against. It does mean that some intraday first-touch events will be labeled as timeouts, but that is preferable to overstating the realizable hit rate. Second, the choice of 63-day volatility for scaling the thresholds stabilizes label proportions across regimes and names. Shorter windows react faster but also allow transient volatility spikes to shrink thresholds and create spurious, low-information touches. With σ_{63} we accept a small loss in responsiveness in exchange for more stationary supervision.

Finally, this section intentionally keeps labels and thresholds conceptually distinct. PT and SL set the difficulty of the task and the horizon defines what the model is asked to learn; the labels record how each attempt actually resolved under those rules. Keeping that separation clear is what lets us reason about coverage, class balance, and calibration which ultimately turns a noisy stream of trade proposals into an estimable, decision-ready probability of success.

VI — Modeling Decisions

VI.1 — Why these learners and what each is meant to learn

We pair LightGBM and a multi-layer perceptron (MLP) because they bring different inductive biases to the same supervision. Gradient-boosted trees excel when signal lives in sparse thresholds, monotone relations, or localized interactions; they are robust to heterogeneous scales and naturally capture piecewise structure that often appears in regime-dependent features. The MLP, by contrast, models smooth nonlinearities and global interactions; with batch normalization and dropout it provides a flexible but regularized function class that can capture probability mass where trees become ragged. Training them on the *same* point-in-time feature set gives us two views of the problem whose errors are only partially correlated. The stacker then learns how to combine these views into a single, well-calibrated probability of success within the quarter-length horizon defined by our labels.

VI.2 — Training protocol and cross-validation

We keep chronology explicit throughout. Fold-1 is used for model discovery: LightGBM is tuned with a compact Bayesian search and trained with time-series cross-validation that includes a gap between train and validation windows to prevent look-ahead. The selected configuration is refit with early stopping and class-balanced weights, using GPU acceleration where available. The MLP follows a nested time-series CV procedure: outer folds provide unbiased performance estimates while inner folds choose depth, width, learning rate, and regularization. During training we construct a balanced input pipeline so each batch sees a roughly even mix of successful, timeout, and bad examples, which stabilizes optimization when class proportions drift through time. Features are standardized for the MLP within each fold to preserve point-in-time integrity; LightGBM consumes the raw engineered features. Both models optimize multiclass cross-entropy, reflecting our three-outcome label space.

VI.3 — From scores to usable probabilities: calibration

Decisions in this framework come from probabilities, not raw scores, so calibration is a first-class step rather than an afterthought. For each model we learn a vector-scaled softmax: a per-class affine transform of the logits followed by a softmax. We fit these transforms only on Fold-2 in a rolling, leakage-free fashion (each validation slice is calibrated using parameters estimated strictly from prior data), and we carry the most recent calibrator into Fold-3. This procedure replaces earlier isotonic/sigmoid or temperature-only approaches and improves multiclass reliability in practice. The result is a set of calibrated class probabilities $(P_{success}, P_{timeout}, P_{bad})$ for every event, which are the quantities we threshold, size from, and pass to the stacker.

VI.4 — The stacking model

Alternative stackers, including an MLP implementation in the experimental branch, were evaluated; the MLP underperformed, while a classwise convex blender delivered the best out-of-sample calibration and log-loss. We therefore replace the neural stacker with a classwise convex blender that is leakage-safe by construction, fits only a handful of parameters, preserves monotonicity in the base probabilities, and proved more stable in Fold-3 ablations. Concretely, we fit the blender on Fold-2 to learn one weight per class that linearly combines the two calibrated distributions; weights are constrained to (0,1) via a sigmoid and lightly regularized toward 0.5 to reduce variance, optimizing multiclass cross-entropy on Fold-2's out-of-sample predictions. At deployment, the learned weights are applied to the Fold-3 calibrated base models via a lightweight inference wrapper. This design keeps the stacker strictly leakage-free, monotone in each base probability, and easy to diagnose as regimes change.

VI.5 — From probabilities to trades

The final output of the stacker is a calibrated probability that a candidate trade will succeed within the 63-day horizon. We admit a trade when this probability exceeds a threshold and clears a margin over the larger of the timeout and bad classes. The margin requirement removes ambiguous setups and explicitly down-weights trades the model believes will drift without resolving. With our strong calibration, we map the success probability to position size to make exposure proportional to conviction; weights are then normalized across names each day, optional volatility targeting and a leverage cap are applied, and asymmetric long/short costs are charged on changes in exposure. Although execution details are treated fully in later sections, the modeling decisions here, namely chronology-respecting CV, leakage-free calibration, and stacking on calibrated distributions, are what make that decision policy statistically meaningful rather than ad hoc.

LightGBM and the MLP provide complementary views of the same quarter-horizon labeling task; careful, time-aware training and explicit calibration turn their scores into reliable probabilities; and a stacker on those calibrated distributions learns when each view is trustworthy. The combination yields a single probability of success that is both interpretable and directly usable for gating and sizing.

VI.6 — Fold-3 ablation (who wins, and by how much)

We compare calibrated LightGBM (CLF_cal), calibrated MLP (MLPv1_cal), and the classwise convex blender (Meta_Blend) on the true OOS slice (Fold-3). The blender achieves the lowest multiclass log-loss (our primary objective) while matching the best one-vs-rest ROC-AUC

Table VI.1 — Fold-3 multiclass log-loss (↓), one-vs-rest ROC-AUC (↑), and accuracy.

Model	Log Loss	ROC AUC	Accuracy
Calibrated LightGBM [CLF_cal]	1.0255	0.5983	0.4488
Calibrated MLP [MLPv1_cal]	1.0309	0.5781	0.4220
Classwise Blender[Meta_Blend]	1.0211	0.5977	0.4447

within noise. Accuracy is included for completeness but is less informative under class drift.

The blender's small but consistent log-loss edge is exactly what we want from a stacker trained on calibrated distributions; it improves decision quality without adding capacity or overfitting risk.

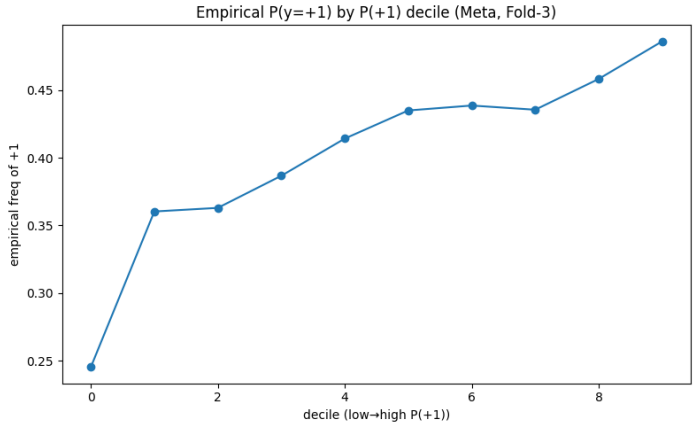

Figure VI.2 — Decile reliability of P(+1) (Fold-3)

We sort Fold-3 events by the model's predicted $P(+1)$, split them into ten equal-count bins, and plot the empirical success rate within each bin. A monotonic up-slope means higher predicted probabilities correspond to higher realized hit rates i.e., the model is both discriminative and well-calibrated in aggregate. With ~165k events on Fold-3, that's roughly $n \approx 16.5k$ observations per decile. In our plot, success frequency climbs from ~0.25 in the lowest decile to ~0.47 in the highest, a healthy spread that justifies using thresholds/margins and mapping probability to size.

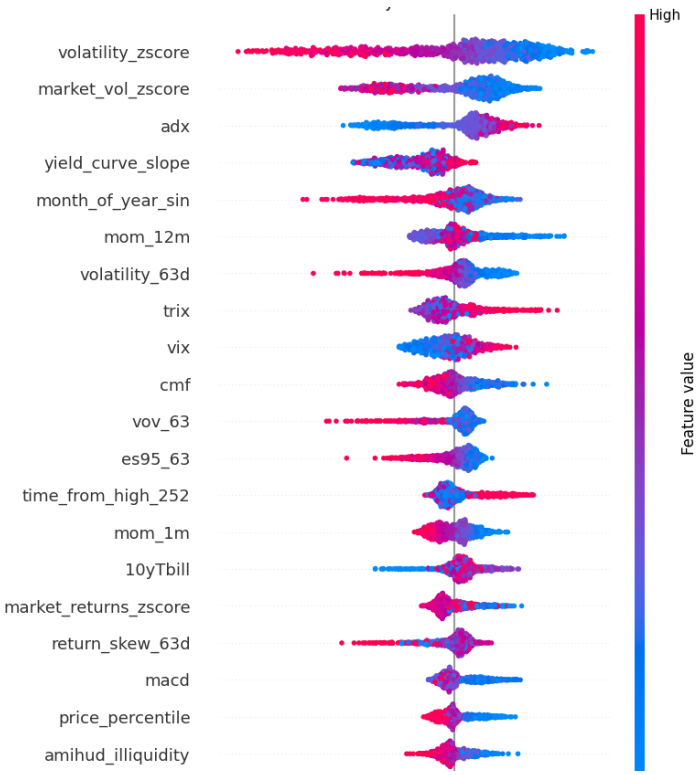

Figure VI.3 – SHAP summary of the LightGBM Classifier for class [+1] (Successful Trades). Features shown are top-k by mean |SHAP|; color encodes feature value (pink=high, blue=low).

SHAP (Shapley Additive exPlanations) attributes to each feature its marginal contribution to the model's output for a given class, here "success" (+1), volatility state dominates; high asset/market vol (pink on the left for volatility_zscore, market_vol_zscore) reduces $P(+1)$; trend strength proxies like adx (pink skewed right) increase $P(+1)$; regime context (e.g., yield-curve slope, VIX level/term) contributes more modest, monotone effects. This aligns with the strategy's intent: the filter favors calm, trending setups and is cautious in stressed regimes.

Together, the decile reliability and SHAP summary justify our threshold with margin gate and probability to size map: the probabilities rank events sensibly, and the drivers behind high $P(+1)$ (low vol, strong trend) are economically coherent.

VI.7 — Calibration quality

Figure VI.4 shows per-class calibration curves for the blender on Fold-3. Curves lie close to the 45° line with mild under-confidence in the extreme bins, desirable for a gate that must avoid over-admitting marginal setups. This supports our use of a hard success threshold plus a margin over the competing class and justifies mapping probability to size when the margin is wide.

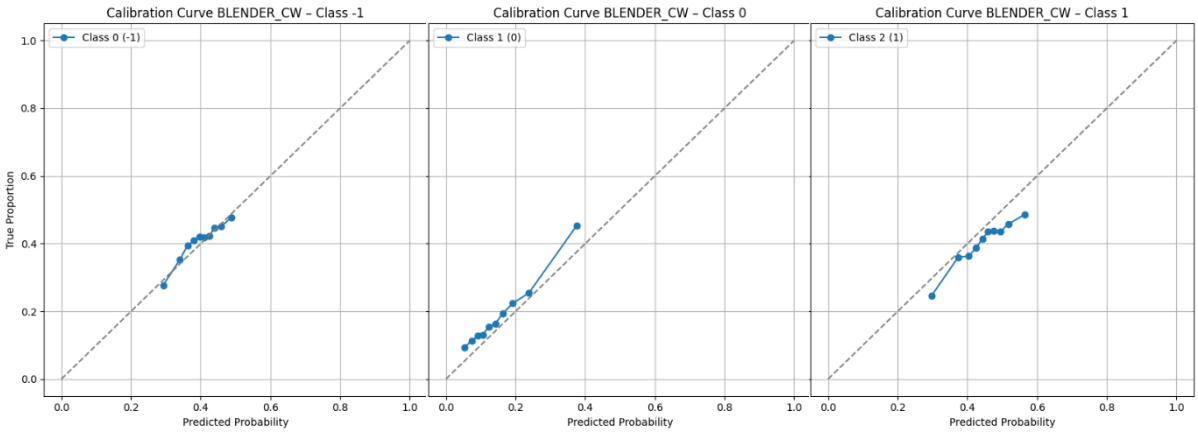

Figure VI.4 — Blender reliability curves, classes {−1, 0, +1}, Fold-3 OOS

All supporting diagnostics are placed in the appendix. Appendix Fig. VI.5a, b show base-model calibration curves for LightGBM and MLP (three classes each). Appendix Fig. VI.6 reports the confusion matrices for the blender, LightGBM, and MLP. Appendix Tab. VI.7 collects feature-importance summaries (LightGBM gain/cover; MLP permutation), and Appendix Tab. VI.8 reproduces the full classification reports. Appendix Fig. VI.9a, b, c present SHAP summaries for MLP across classes (−1, 0, +1); Appendix Fig. VI.10a, b provide the remaining LightGBM SHAP summaries (classes 0 and −1). Finally, Appendix Fig. VI.11 & Fig VI.12 contain the two meta-vs-base diagnostics: the Fold-3 ΔNLL histogram (meta − best base) and the P(+1) scatter (MLP vs blender), which illustrate where the blender improves probability mass.

VII — Execution & Portfolio Construction

VII.1 — Timing and return alignment

All PnL is measured "close → next close." Target weights set at time *t* are applied to the forward return at *t + 1*, and the same timing is used to compute the realized volatility that drives vol targeting. Concretely, the sizing routine forms raw portfolio returns as $\sum_i \omega_{t,i} r_{t+1,i}$, i.e., it multiplies end-of-day target weights by next-day returns. Realized volatility is then the EWMA(63) of that forward PnL stream, annualized by $\sqrt{252}$, so the risk control reacts to the same return that produces PnL. This closes the loop that occasionally drifts in research code (e.g., sizing to *today's* return but reporting *tomorrow's* PnL); here both are explicitly aligned via returns.shift(-1) inside the sizer. A certain number of features can be deactivated in config by setting their value to -1, notably MIN_GAP, TOP_K_PER_DAY, TARGET_VOL and LEVERAGE_CAP.

VII.2 — From probabilities to target weights

The stacker outputs a calibrated distribution over $\{+1,0,-1\}$. We first gate each candidate on the success probability using side-specific thresholds and a minimum margin over the competing classes; only events that pass both tests enter the ranking stage. When daily breadth still exceeds capacity, we apply a combined Top-K across both sides (no per-side quota). With META_SCORE_MODE="edge", the ranking score is the excess probability over the side's threshold, $score_i = P_{success}(i) - \theta_{side}(i)$ where $\theta_{side} = 0.45$ for longs and 0.5 for shorts; the logit_edge variant applies the same idea in log-odds space. This ranking compares $P_{success}$ to the *acceptance threshold*, that's distinct from the margin check used earlier to admit trades which compares $P_{success}$ to the best competing class $max\{P_{timeout}, P_{bad}\}$. We retain the K names with the largest scores each day regardless of sign, which uses breadth efficiently and avoids per-side starvation.

In weight construction, accepted names are then sized with margin weighting $\omega_i \propto s_i \cdot \max\{0, P_{Success}(i) - \max\{P_{timeout}(i), P_{bad}(i)\}\}$, with $s_i \in \{+1,-1\}$the base side, normalized so that $\sum_i |\omega_i| = 1$ before risk overlays. Alternatives are "prob" $s_i \cdot P_{Sucess}$ and "odds", a bounded log-odds map. This is separate from the Top-K score above.

VII.3 — Churn controls and pathwise execution

We dampen day-to-day oscillations with a *λ-blend* (LAMBDA_BLEND = 0.2 in config) applied to the *final* target weights, not to raw scores. The update is $\omega_t^{blend} = (1-\lambda)\omega_{t-1}^{blend} + \lambda\omega_t^*$, with $\lambda = 0.2$ in our runs. This adds inertia to weights without biasing selection and reduces needless rebalancing in flat regimes. After blending, we apply a pathwise *minimum trade* (MIN_TRADE_EPS = 0.01) filter that suppresses micro-adjustments: if the step $\left|\omega_t^{blend} - \omega_{t-1}^{exec}\right|$ is below 100 bps, we keep the previous executed weight for that name. The executed

book is then pruned below the same epsilon to zero out dust positions. Together these two guards act as our *hysteresis layer*, they absorb small noise and require ~1% moves before weights change.

VII.4 — Vol targeting and leverage cap

Risk is controlled in two layers. First, we scale the entire book to an annualized volatility target (TARGET_VOL = 0.2) using the EWMA(63) of *forward* portfolio PnL (Section VII.1). The scaler is applied with a one-day lag and a touch of inertia to avoid pro-cyclical whipsaws, then weights are re-normalized. Second, we cap gross leverage to a hard ceiling (LEVERAGE_CAP = 3) by shrinking the book proportionally on days when the scaled sum of absolute weights would otherwise exceed the cap. Both overlays are portfolio-level transforms and therefore preserve cross-sectional stock selection.

VII.5 — Transaction costs and executed PnL

Costs are charged on changes in the *executed* book, computed separately for long and short legs to avoid the classic flip double-count. We decompose the path of executed weights into long increases and short increases and apply asymmetric bps charges (LONG_SIDE_TC = 10 bps; SHORT_SIDE_TC = 20 bps) then subtract the daily cost from forward PnL. The per-side construction ensures that a long → short flip is charged once for closing the long and once for opening the short, rather than twice on the same notional, and that same-day round-trips created by tiny oscillations are removed by the pathwise min-trade filter upstream.

VII.6 — Implementation notes and fixes

Two issues that commonly erode paper alpha were addressed explicitly in code. First, vol targeting, PnL, and turnover are all computed off the same "close → next close" executed weights and returns, eliminating subtle timing drifts that can otherwise flatter backtests. Second, we moved turnover and costs to the *executed* book with a per-side decomposition, removing an earlier double-count that can occur if one measures turnover on target weights or treats flips as two full trades. These choices are visible in the sizer's construction of forward PnL, the EWMA risk estimator, the λ-blend/hysteresis layer, and the per-side turnover logic, and they are parameterized in config for reproducibility and audit.

VIII — Evaluation Strategy

VIII.1 — Out-of-sample protocol and leakage control

In time-series ML, any decision at time t must be based only on information available at or before t; strict, chronological fold separation (with gaps where needed) prevents label/feature leakage and keeps model selection independent from final evaluation.

We separate discovery, calibration, and evaluation chronologically: Fold-1 is used to select model families and hyperparameters; Fold-2 is held out for calibration and for fitting the classwise convex blender on base-model probabilities; Fold-3 is the true out-of-sample period. All headline results (tables and charts) are reported on Fold-3 net returns after costs and overlays. Any metric computed on Fold-1/2 is diagnostic only (to choose models or verify calibration), not used to claim performance.

VIII.2 — Portfolio-level metrics

Performance is computed from the executed book (post λ-blend, min-trade filter, vol targeting, leverage cap) against next-day returns. We report annualized return and volatility, Sharpe and Sortino, max/average drawdown with duration, CVaR (mean of the worst 5% daily returns), skew and kurtosis, correlation to SPY, and regression alpha–beta versus SPY. These statistics are produced on net returns (after 10 bps long / 20 bps short, charged on per-side turnover) so the risk/return profile reflects realistic frictions.

VIII.3 — Calibration and classification diagnostics

Because gating and sizing are probability-driven, we evaluate statistical quality alongside PnL. For each base model and for the blender we compute multiclass log-loss and ROC-AUC, confusion matrices and reliability curves per class. We summarize ECE (mean absolute calibration error from quantile-binned calibration curves) and Brier scores per class to quantify how well predicted probabilities match empirical frequencies. These diagnostics are produced on Fold-2 (where calibration is learned) and validated on Fold-3 to ensure that thresholding and margin rules rest on well-behaved probabilities.

VIII.4 — Benchmarks and portfolio variants

Results are framed against three comparators. First, SPY as a market baseline. Second, a standard momentum benchmark built from the same PIT universe and rebalanced on the same daily schedule (long-only and long–short variants), which isolates the value of the meta-filter relative to a conventional momentum allocator. Third, two portfolio combinations that speak to deployability on a multi-asset desk: a 50/50 blend (Strategy net + SPY) and a beta-neutral variant constructed by regressing strategy returns on SPY each day and hedging with $\hat{\beta}$. We report the same metrics for these variants (including realized beta and rolling correlation) so

the reader can see how the strategy improves a core equity sleeve and how it behaves in a market-neutral context.

VIII.5 — Rebalancing, frictions, and realism

The backtest rebalances daily. Target weights at t are applied to returns at $t + 1$; the same timing governs volatility targeting (EWMA-63 of forward PnL), cost accrual, and turnover measurement. Costs are charged on changes in the executed book with a per-side decomposition to avoid flip double-counting. The ranking capacity constraint is implemented as a combined Top-K across both sides (no per-side quotas), which better reflects real breadth and avoids empty books on one side. All overlays (λ-blend, min-trade, vol target, leverage cap) are applied after selection and weight mapping, and all primary figures are reported after these overlays and costs.

VIII.6 — Reporting artifacts and reproducibility

For each run we generate a compact performance table (Strategy gross/net, 50/50 net, beta-neutral net, SPY, and momentum comparators) and a standard figure set: cumulative returns (with SPY and the 50/50 blend), underwater/drawdown, rolling Sharpe, rolling correlation to SPY, and alpha–beta scatter with regression fit. Calibration plots (per-class reliability curves), confusion matrices, and per-trade summaries (hit rates, profit factor, holding-time distribution) are also saved. Every artifact is tied back to the manifest and config snapshot, so figures can be reproduced exactly from the recorded folds, label spec, hyperparameters, and blender weights.

IX — Key Results

IX.1 — Performance at a glance (net of costs)

Over the full out-of-sample window, the meta-labeled strategy delivered 12.17% annualized return at 19.51% vol for a Sharpe of 0.62 (Sortino 0.80). The 50/50 blend with SPY achieved 14.64% at 13.43% vol for a Sharpe of 1.09, materially above SPY's 0.69 (14.48% at 20.99% vol), with a similar max drawdown (–28.07% vs SPY –33.72%). A beta-neutral variant posted 14.20% at 19.36% vol for a Sharpe of 0.73. These results are net of asymmetric costs (10/20 bps for long/short turnover), vol-targeting and leverage controls.

Across 3,612 executed trades, the win rate is 65.75% (profit factor 2.92), with +1 successes 2,078, –1 failures 1,145, and timeouts 389. Average holding time is 24.2 days, consistent with the 63-day triple-barrier horizon and Top-K/threshold gating. Longs and shorts both contribute (hit rates 57.43% and 61.29%, respectively). Average PnL per trade is 0.84% (median 0.38%); notional-weighted win rate is 65.81%.

Table IX.1 — Final summary : Annualized return/vol from daily net returns; vol uses √252; Sortino uses semi-vol on negative days; CVaR is mean of worst 5% daily returns; alpha–beta from OLS of strategy on SPY (daily), intercept annualized; correlation to SPY; drawdown duration is longest peak-to-trough in days. Trade stats (counts, win rate, PF, holding period, side hit-rates) come from realized executed trades (close → next-close), not target weights.

	Meta-Labeling (Gross)	Meta-Labeling (Net)	50% Blend (Net)	Beta Neutral (Net)	SPY	Std Momentum
Annualized Return	16.74%	12.17%	14.64%	14.20%	14.48%	14.44%
Annualized Vol	19.50%	19.51%	13.43%	19.36%	20.99%	24.17%
Semi-volatility	15.14%	15.13%	10.17%	14.92%	17.16%	19.39%
Sharpe Ratio	0.86	0.62	1.09	0.73	0.69	0.60
Sortino Ratio	1.11	0.80	1.44	0.95	0.84	0.75
Conditional VaR (CVaR)	-2.95%	-2.97%	-2.00%	-2.94%	-3.20%	-3.56%
Max Drawdown	-27.32%	-28.07%	-28.07%	-30.60%	-33.72%	-37.83%
Avg Drawdown	-6.07%	-7.05%	-4.14%	-6.82%	-6.25%	-5.29%
Max DD Duration (days)	297	305	211	301	488	429
Skew	-0.700	-0.702	-0.553	-0.729	-0.544	-0.596
Kurtosis	4.657	4.647	3.803	4.147	11.511	12.953
Correlation to SPY	-0.122	-0.121	0.693	0.000	1.000	0.903
Alpha	0.192	0.152	0.076	0.152	-0.000	0.001
Beta	-0.113	-0.113	0.444	0.000	1.000	1.040
Positive Months	38	37	38	38	39	38
Negative Months	21	22	22	22	21	22
Trade Count	3612	3612	N/A	N/A	N/A	N/A
Win Rate (WR)	65.75%	65.75%	N/A	N/A	N/A	N/A
Successful (+1) Trades	2078	2078	N/A	N/A	N/A	N/A
Bad (-1) Trades	1145	1145	N/A	N/A	N/A	N/A
Timeout (0) Trades	389	389	N/A	N/A	N/A	N/A
Avg Holding Time (days)	24.2	24.2	N/A	N/A	N/A	N/A
Notional-Weighted WR	65.81%	65.81%	N/A	N/A	N/A	N/A
Avg PnL per Trade	0.84%	0.84%	N/A	N/A	N/A	N/A
Median PnL per Trade	0.38%	0.38%	N/A	N/A	N/A	N/A
Profit Factor	2.92	2.92	N/A	N/A	N/A	N/A
Long Hit Rate	57.43%	57.43%	N/A	N/A	N/A	N/A
Short Hit Rate	61.29%	61.29%	N/A	N/A	N/A	N/A

Note on N/A entries: Some fields (trade count, win rate, holding time, profit factor) exist only for the meta-labeled sleeve, which has explicit trades; the 50/50 blend, beta-neutral, SPY, and standard momentum columns are just return series, so those trade-level stats are not applicable. Also, risk and tail metrics for the blend/hedge can't be inferred from the sleeve alone: they

depend on the joint path and co-movement with SPY, as well as costs and overlays applied to the combined series. In short, the N/As are intentional, and the non-N/A numbers come from the realized daily returns of each portfolio.

IX.2 — Diversification and beta profile

Daily alpha–beta regression against SPY shows a negative market exposure for the strategy, $\beta \approx -0.11$ with $\alpha \approx 0.15\%$ annualized and an overall correlation of –0.12. This negative beta plus low correlation is the main reason the 50/50 blend sharpens risk-adjusted returns and cuts volatility.

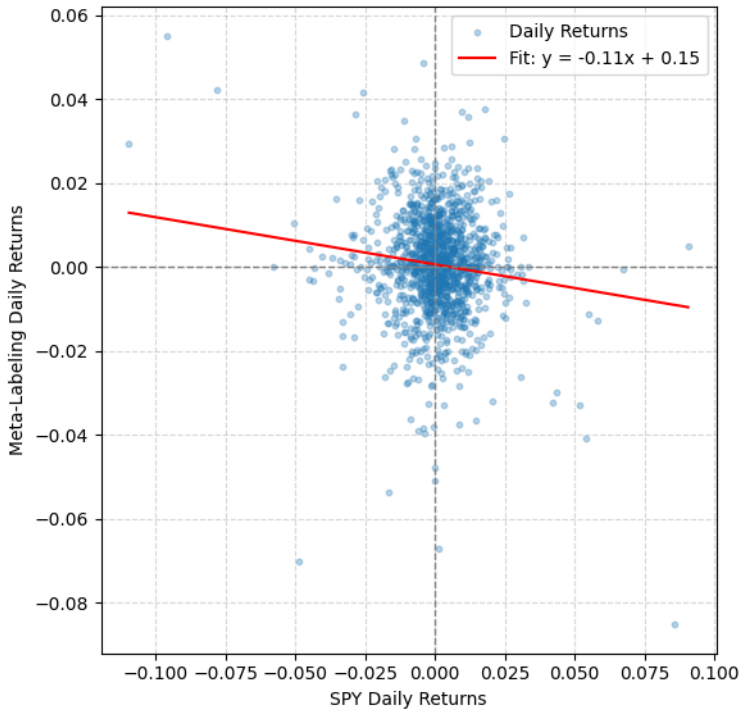

Figure IX.2 — Alpha/Beta scatter of meta labeling strategy vs SPY with fit line

IX.3 — Cumulative return and drawdowns

Cumulative net performance tracks above SPY after 2020's shock and compounds steadily through multiple regimes; standard cross-sectional momentum (unfiltered) co-moves with SPY and suffers deeper swings. The strategy's max drawdown is –28.07% (avg –7.05%, max duration 305 days), while SPY's is –33.72% (avg –6.25%, duration 488).

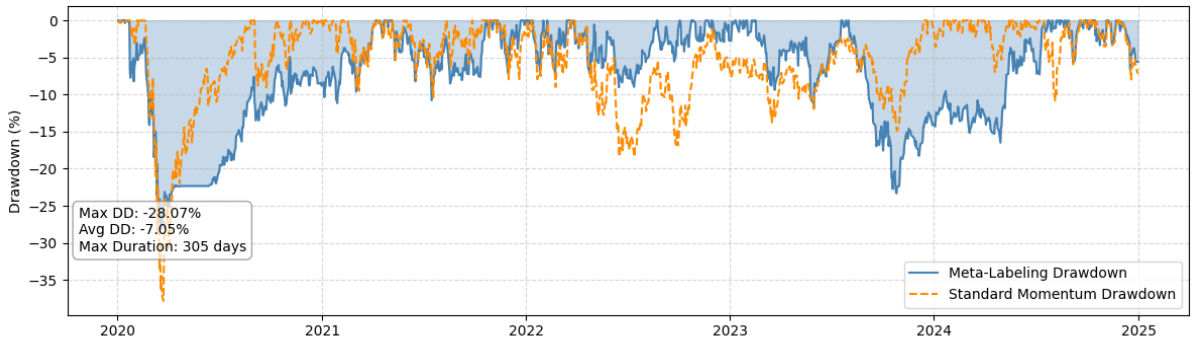

Figure IX.3 — Underwater Drawdown of meta labeling strategy vs Momentum

Figure IX.4 — Cumulative returns of meta labeling strategy (Meta gross & net, SPY, Momentum)

IX.4 — Rolling behavior and portfolio fit

Rolling 20-day correlations to SPY hover around zero with excursions negative during stress, reinforcing the blend rationale; rolling 60-day Sharpe shows episodic spikes when the filter is selective and flat when breadth is thin.

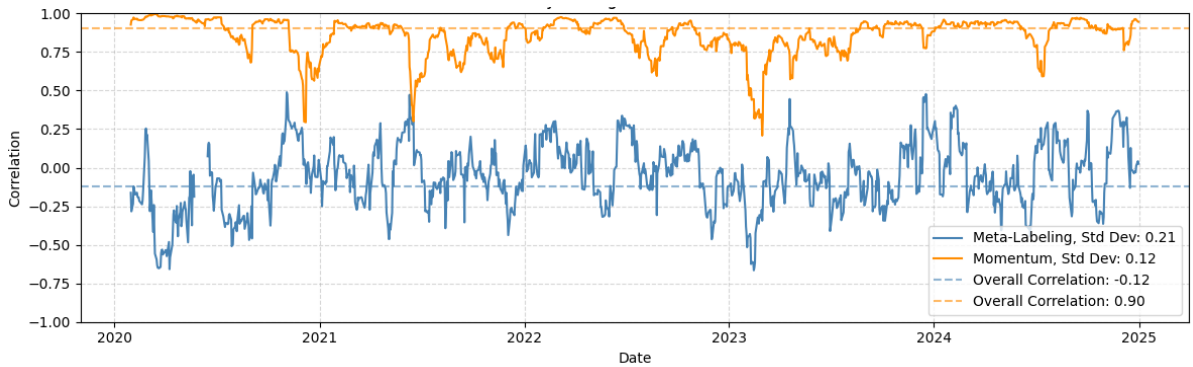

Figure IX.5 — 20-day rolling correlation of meta labeling strategy

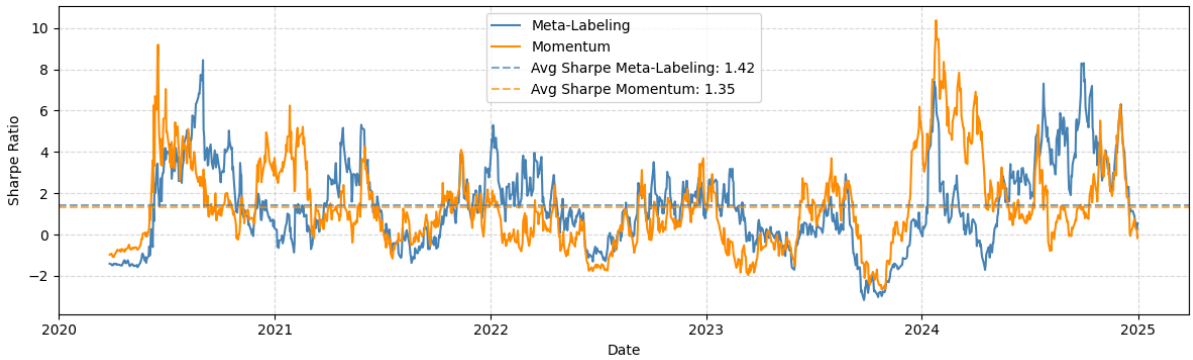

Figure IX.6 — 60-day rolling Sharpe of meta labeling strategy

IX.5 — Execution realism: turnover and leverage

Pathwise turnover averages ~15%/day with a 5-day MA peak near 0.52, largely contained by λ-blend (0.2) and the 1% min-trade rule. Gross leverage averages ~0.86×, peaking below 2.0×, comfortably inside the 3× cap; net exposure averages ~0.80×.

IX.6 — Interpretation

The meta-filter behaves as intended: it selects fewer, better trades (Top-K + gap), keeps portfolio beta slightly negative, and translates calibrated probabilities into stable PnL once overlaid with conservative execution (EWMA-vol targeting, leverage cap, per-side costs). The pure strategy is already diversifying ($\beta \approx -0.11$, $\rho \approx -0.12$). The practical way to deploy is the 50/50 blend with SPY, which raises Sharpe to 1.09 while halving volatility relative to the pure meta strategy and trimming drawdown versus holding the market alone.

To keep the section readable, we show the strategy-level diagnostics here (performance summary Tab. IX.1, alpha–beta fit Fig. IX.2, underwater drawdowns Fig. IX.3, cumulative returns Fig. IX.4, rolling correlation Fig. IX.5, rolling Sharpe Fig. IX.6). All variant and operational plots are collected in the appendix: the full figure sets for the 50/50 blend and for the beta-neutral sleeve (each including alpha–beta regression Fig. IX.9, 14 , cumulative curve Fig. IX.10, 15, underwater Fig. IX.11, 16, rolling correlation Fig. IX.12, 17, and rolling Sharpe Fig. IX.13, 18), and the strategy's execution diagnostics (5-day average turnover Fig. IX.7 and daily gross leverage Fig. IX.8).

X — Implementation Review:

X.1 — Challenges & Fixes

X.1.1 — Data & Reproducibility

X.1.1.1 — Vendor survivorship and availability gaps

Problem: Yahoo finance omits many delisted names; even with a PIT universe this can create availability bias and unstable cross-section size. Fix: Built an explicit PIT mask; excluded zero-availability tickers; traded partially available names only on dates with both PIT membership and data. Evidence: Availability dashboard and time-series of cross-section size; average ticker availability ≈ 76% within PIT windows; missing/availability reports saved alongside the run.

X.1.1.2 — Price anomalies and feed glitches

Problem: Spurious zero/near-zero prices and implausible jumps polluted returns and could trip barriers spuriously. Fix: Nulled non-sensical prices; set |1-day return| > 500% to NaN; logged offenders rather than winsorizing so downstream handling stays explicit. Evidence: "Spike" log and before/after counts; no barrier touches attributable to single-tick artifacts in spot checks.

X.1.1.3 — Macro inputs and reproducibility

Problem: External series (10Y, 10Y–3M, PIT constituents) can drift when pulled live, breaking parity with prior runs. Fix: Manually snapshotted inputs at project start; checksummed in the run manifest; loaders forward-fill to the trading calendar deterministically. Evidence: manifest.json with file hashes; config_snapshot.json and fold fingerprints tie figures in the report to exact inputs.

X.1.2 — Labeling & Evaluation

X.1.2.1 — Label horizon and the "last day" effect

Problem: Events near the sample end lacked a full 63-day horizon, inflating timeouts and skewing class balance. Fix: Dropped events whose vertical barrier would extend beyond data availability; labels computed on closes only (no intraday look-through). Evidence: Consistent label mix across folds; holding-period distribution without terminal spikes.

X.1.2.2 — Probability calibration and class-index hygiene

Problem: Inconsistent class ordering and small calibration drifts made thresholds brittle. Fix: Locked class order {−1, 0, +1} across data, models, and metrics; applied vector-scaled softmax calibration on a leakage-free Fold-2 and carried the latest calibrator into Fold-3. Evidence: Blender reduces multiclass log-loss vs. best base on Fold-3; reliability curves near 45°; decile-lift monotonic.

X.1.3 — Execution & Portfolio Construction

X.1.3.1 — Timing drift between sizing and PnL

Problem: Early prototypes sized on contemporaneous returns but reported next-day PnL, flattering results. Fix: Aligned everything to close → next-close: targets at ttt applied to returns at $t + 1$; EWMA(63) vol targeting computed from the same forward PnL stream. Evidence: Codepath uses returns.shift(-1) in sizer; realized vol estimated on forward PnL; diagnostics match this timing.

X.1.3.2 — Turnover and cost double-count on flips

Problem**:** Measuring turnover on targets and/or not separating long/short changes double-counted long ↔ short flips. Fix: Charged costs on executed paths only, decomposed per-side (10 bps long, 20 bps short); added pathwise min-trade (100 bps) to suppress micro-round-trips. Evidence: Turnover and cost series drop to realistic levels; flip days show one close + one open, not two full tickets.

X.1.3.3 — Admission logic not enforced uniformly

Problem: Early runs occasionally admitted trades that cleared the success threshold but failed the margin (gap) test, or vice-versa. Fix: Centralized gating: side-aware thresholds (long 0.45, short 0.50) and a 0.10 margin vs. max{timeout, bad} must both pass before ranking; unit tests on edge-cases. Evidence: Daily "eligible count" logs; no violations in spot audits of top-K selections.

X.1.3.4 — Per-side top-K starvation

Problem: Per-side quotas produced degenerate days (e.g., few qualified shorts) and wasted breadth. Fix: Switched to combined Top-K across both sides (K=3) ranked by an "edge" score relative to the side's threshold; preserved side-aware acceptance for admission. Evidence: More consistent breadth utilization; fewer zero-side days; same K at lower churn.

X.1.3.5 — Weight normalization and leverage discipline

Problem: Non-normalized targets and missing risk overlays led to noisy gross exposure and spiky turnover. Fix: Normalized absolute weights to 1.0 pre-risk; added λ-blend (0.2) and min-trade; applied EWMA(63) vol targeting to 20% and a 3× gross leverage cap. Evidence: Daily leverage track stays well below the cap (appendix); turnover stabilizes without loss of hit-rate.

X.1.4 — Practical Constraints

X.1.4.1 — Runtime and memory constraints

Problem: Limited RAM/VRAM and no dedicated rig constrained search depth and diagnostics. Fix: Cached PIT artifacts to parquet; pruned highly collinear features early; used compact Bayesian search for LGBM; batched MLP training with early stopping; sampled observations for SHAP and stored summaries. Evidence: End-to-end runs complete within hardware limits; reproducible across folds with identical manifests.

X.2 — Limitations

X.2.1 — Data fidelity and universe coverage

This study is intentionally built on free, widely reproducible sources. That decision comes with real constraints. Yahoo Finance does not preserve many delisted histories; even with a point-in-time (PIT) membership mask, historical depth is uneven and availability averages ~76% within PIT windows. Prices are close-to-close only; there is no trade/quote microstructure, no institutional corporate-actions feed, and no PIT fundamentals. These choices keep the project portable and auditable, but they cap the ceiling on what the base alpha and the meta-filter can jointly achieve. A production deployment would replace the price feed with a vendor that retains delisted series and corporate action events (and add PIT fundamentals and borrow/short data).

X.2.2 — Label design and horizon coupling

Our supervision target is a triple-barrier outcome over a quarter-length horizon (63 trading days) with barriers scaled by σ_{63}, while portfolio risk overlays use EWMA(63) on forward PnL, by design, they serve different roles. This matches the monthly momentum base but ties "good/bad" to a single time scale: fast edges that resolve in days and slow, fundamental theses can be undercounted even if economically sound. If other speeds are required, we would retune PT/SL and t_{max} and reshape features; multi-horizon labels or regime-conditional barriers are natural extensions (see X.3.3 and X.3.5).

X.2.3 — Modeling and calibration caveats

The stacker is a classwise convex blender fit on calibrated base probabilities (LightGBM, MLPv1). We chose it because it beat an MLPv2 stacker out-of-sample on log-loss/ECE and proved more stable and easier to re-calibrate, not merely because it is "simple." Calibration is learned on Fold-2 via vector-scaled softmax and carried into Fold-3; in practice, one would schedule rolling recalibration and monitor reliability/decile monotonicity to catch drift.

X.2.4 — Execution realism, capacity, and costs

Backtests include asymmetric costs on executed turnover (10/20 bps long/short), EWMA(63) vol targeting to 20% and a 3× gross-leverage cap. That's first-order realism; it omits impact, borrow fees/availability, locates, and participation-rate constraints. No backtest is ever complete ("Even if your backtest is flawless, it is probably wrong" De Prado (2018)[3]); the right next step is a period of paper/live-shadow trading with borrow and participation limits enforced, then iterating with impact models (e.g., square-root with volume scaling) and borrow-fee scenarios.

[3] Advances in Financial Machine Learning – M.L. De Prado (2018)

X.3 — Extensions

The items below are optional research ideas, not commitments. Several are already partially implemented (e.g., probability-aware sizing with a margin map; EWMA overlays). We list them to show plausible paths a desk might explore, not to diminish the current build.

X.3.1 — Higher-fidelity data

What it adds: Delisted price history, PIT fundamentals, and borrow/short fields raise supervision quality and improve regime features. How to try: Swap the price vendor for one that retains delistings; add PIT fundamentals (e.g., value/quality ratios) and basic borrow flags; keep the current PIT mask and manifest discipline. Success test: More stable label mix across regimes, stronger calibration (lower ECE/Brier), and higher net performance at the same turnover.

X.3.2 — Side-aware modeling

What it adds: Long and short setups are not symmetric; modeling that asymmetry can sharpen acceptance. How to try: (i) Two calibrated learners, one per side, with per-side thresholds; or (ii) a 5-class direction×outcome target if supervision stays dense. Keep the classwise blender. Success test: Lower log-loss and better reliability per side; cleaner gap between accepted and rejected trades, especially on shorts.

X.3.3 — Adaptive acceptance

What it adds: Gates that adapt to market state reduce over-trading in stress and under-trading in calm. How to try: A small state machine keyed to volatility/crowding: widen margin and raise thresholds in high-vol regimes; relax in low-vol; allow K to vary within bounds. Add hysteresis to avoid flapping. Success test: Higher Sharpe at equal or lower turnover; acceptance rate that co-moves sensibly with regime diagnostics.

X.3.4 — Probability → size mappings (margin is the default)

What it adds: Alternate maps can trade off conviction vs. churn more efficiently. How to try: Compare our margin map to (i) a bounded log-odds map and (ii) a simple expected-utility map that subtracts a turnover penalty. Enforce exposure monotonicity and daily $\sum | w |= 1$. Success test: Higher net return per unit turnover and stable gross exposure without adding tail risk.

X.3.5 — Barrier σ alternatives (labels only)

What it adds: More stationary barrier difficulty across regimes. How to try: For triple-barrier scaling only, replace rolling σ with EWMA(63) or a range-based estimator (Parkinson/Garman–Klass); re-run the PT/SL grid and horizon sweep. Success test: Flatter class proportions through time and improved calibration without sacrificing coverage.

X.3.6 — Fundamentals sleeve (value or quality)

What it adds: A "structural" base edge lets the meta-filter learn when fundamental theses realize. How to try: Build a PIT value/quality signal (e.g., EBIT/EV, accruals, margins), then apply the

same meta-filter pipeline and gates. Success test: Better selectivity (higher win rate and profit factor) at similar capacity, and complementary correlation to SPY.

X.3.7 — Microstructure sleeve (short-horizon)

What it adds: A test of the framework where costs dominate unless entry states are very specific. How to try: Intraday mean-reversion or event-driven base signal with TAQ/L2 features (queue imbalance, realized vol, short-term order flow), shorter t_{max} (e.g., 30–90 minutes), and an explicit fee/impact model. Success test: Positive net after costs with a clear acceptance gap; sharply reduced trade count relative to base.

X.3.8 — Execution realism

What it adds: Capacity-aware performance and more credible net metrics. How to try: Add square-root impact with volume scaling, borrow fees/HTB exclusions, participation caps, and alternate clocks (open/close/VWAP). Follow with a paper-trading period; no backtest is ever "complete." Success test: Stable performance when moving from backtest → paper; explicit capacity curves (PnL vs. participation).

X.3.9 — Learning-to-rank / listwise selection

What it adds: Selects the set that makes the best portfolio, not just the best individual names. How to try: Learn a day-level ordering that maximizes a differentiable Sharpe-like surrogate minus a turnover penalty, then take Top-K under the usual capacity guardrails. Keep close to current edge scores via a small regularizer. Success test: Higher OOS Sharpe at the same K and turnover; improved diversification within daily picks.

X.3.10 — Conformal acceptance

What it adds: A coverage-controlled, data-driven gate on top of calibrated probabilities. How to try: Define non-conformity $s = 1 - P(success)$; on a rolling window of true winners, take the $(1 - \alpha)$ quantile of s and accept new trades only if s is below it. Use alongside the current threshold + margin gate. Success test: Acceptance rates that adapt to regime while keeping realized win rates above a target; fewer ambiguous trades admitted.

X.3.11 — Policy learning for sizing

What it adds: Lets data choose among a few sizing behaviors that balance edge and churn. How to try: Start as a contextual bandit: discretize sizes (e.g., {0.5×, 1.0×, 1.5× of margin weight}); features are calibrated probs + churn/risk signals; reward is next-day net PnL minus a turnover penalty. Evaluate offline with robust CV. Success test: Net PnL improves after costs without raising turnover or tail risk; policy remains exposure-monotone.

To sum up, the current build favors stability, calibration, and reproducibility. The most impactful next steps are higher-fidelity data, side-aware learning, and execution realism; the ranking, conformal, and policy-sizing ideas are lightweight add-ons that can be tested quickly and kept only if they clear an OOS bar.

XI — Conclusion

This project sets out to turn a noisy momentum allocator into a disciplined, probability-aware process. The meta-labeling filter does exactly that: it admits fewer, better setups and converts calibrated success probabilities into stable portfolio weights. Out-of-sample, the sleeve stands on its own as a diversifier (low, slightly negative beta and correlation to SPY) and, more importantly for deployment, it improves a core equity sleeve when blended. A simple 50/50 mix with SPY raises risk-adjusted returns while holding drawdowns in check, which is precisely the profile a multi-asset desk looks for when adding a systematic satellite to a benchmarked book. Trade-level diagnostics, namely win rate, profit factor, and a quarter-length holding time, align with the filter's intent rather than relying on fragile parameter choices.

The path to that outcome is deliberately orthodox. Labels are defined by a triple-barrier contract at a 63-day horizon with volatility-scaled targets and stops; features are measured strictly at decision time under a point-in-time universe; base learners (LightGBM and MLP) are trained with time-aware CV and then calibrated via vector-scaled softmax; a classwise convex blender combines calibrated distributions into one usable probability of success. Gating, ranking, and sizing are monotone and interpretable (side-aware thresholds, a margin over competing classes, Top-K, and a margin-based weight map). Execution is tied to a realistic clock: close → next-close returns, per-side costs on executed turnover, EWMA-based vol targeting, and a gross-leverage cap, so the reported net performance reflects the way a live sleeve would actually trade. Reproducibility is enforced with manifests, config snapshots, and fold fingerprints so that every figure in the report can be tied back to exact inputs and parameters.

No backtest is final, but this one is deployable. The most credible way forward is a period of live shadow trading with borrow and participation limits enforced, alongside a data upgrade that retains delisted histories and, where appropriate, adds PIT fundamentals or microstructure fields. Those enhancements should widen the set of base edges the filter can supervise without changing the core logic that makes it robust: clean labels, calibrated probabilities, conservative execution, and transparent controls. In short, the build demonstrates that a well-calibrated meta-filter can convert a standard momentum signal into a professional trading sleeve that fits neatly into a real portfolio and earns its keep by being selective when it matters.

XII — Appendix

III – Pipeline Overview

Table III.1 – Pipeline organization per stage, indicating inputs and outputs with the associated module

Stage	Inputs → Outputs	Key module(s)
PIT data build	yfinance OHLCV, constituents → PIT-filtered prices/volumes, SPY, VIX (parquet)	data_download.py
Loaders	Parquets/CSVs → aligned prices, returns, VIX, rates (PIT masks applied)	data_loader.py
Base strategy	Monthly momentum ranks → daily long/short participation signals + baseline returns	strategy.py
Labeling	Signals + daily closes + σ_{63} → Y in {+1 success, 0 timeout, −1 bad} + (t_0,t_1) using triple-barrier with PT=5σ, SL=5σ, t_{max}=63	labeling.py
Feature build	Prices/returns/volumes/VIX/rates/TA → X (pruned, PIT-aligned) matched to Y	features.py
Layer-1 models (discovery)	X_1, Y_1 (Fold-1) → tuned LGBM, MLP (TS-CV w/ gap; class-balanced; early stopping; GPU if avail.)	modeling.py, mlp_modeling.py
Fold-2 inference & calibration	X_2, Y_2 (Fold-2) + tuned models → calibrated multiclass probabilities (vector-scaled softmax; rolling, leakage-free)	modeling.py, mlp_modeling.py
Layer-2 stacker	$X_{meta,2}$, Y_2 (Fold-2) → Classwise Convex Blender (per-class weights over calibrated LGBM/MLP)	mlp_modeling.py
Model analysis	LGBM/MLP/Stacked MLP → AUC/log-loss, calibration curves/ECE/Brier, confusion matrices, SHAP/importance	analysis.py
Filtering & sizing	Stacker probabilities + threshold + margin → filtered signals; optional prob-sizing; daily normalization, vol targeting, leverage cap; asymmetric long/short costs	signals.py, sizing.py
OOS backtest	Weights + returns (Fold-3) → net PnL, risk, diagnostics, plots	evaluation.py
Orchestration	Runs all stages with folds & configs	main.py, config.py

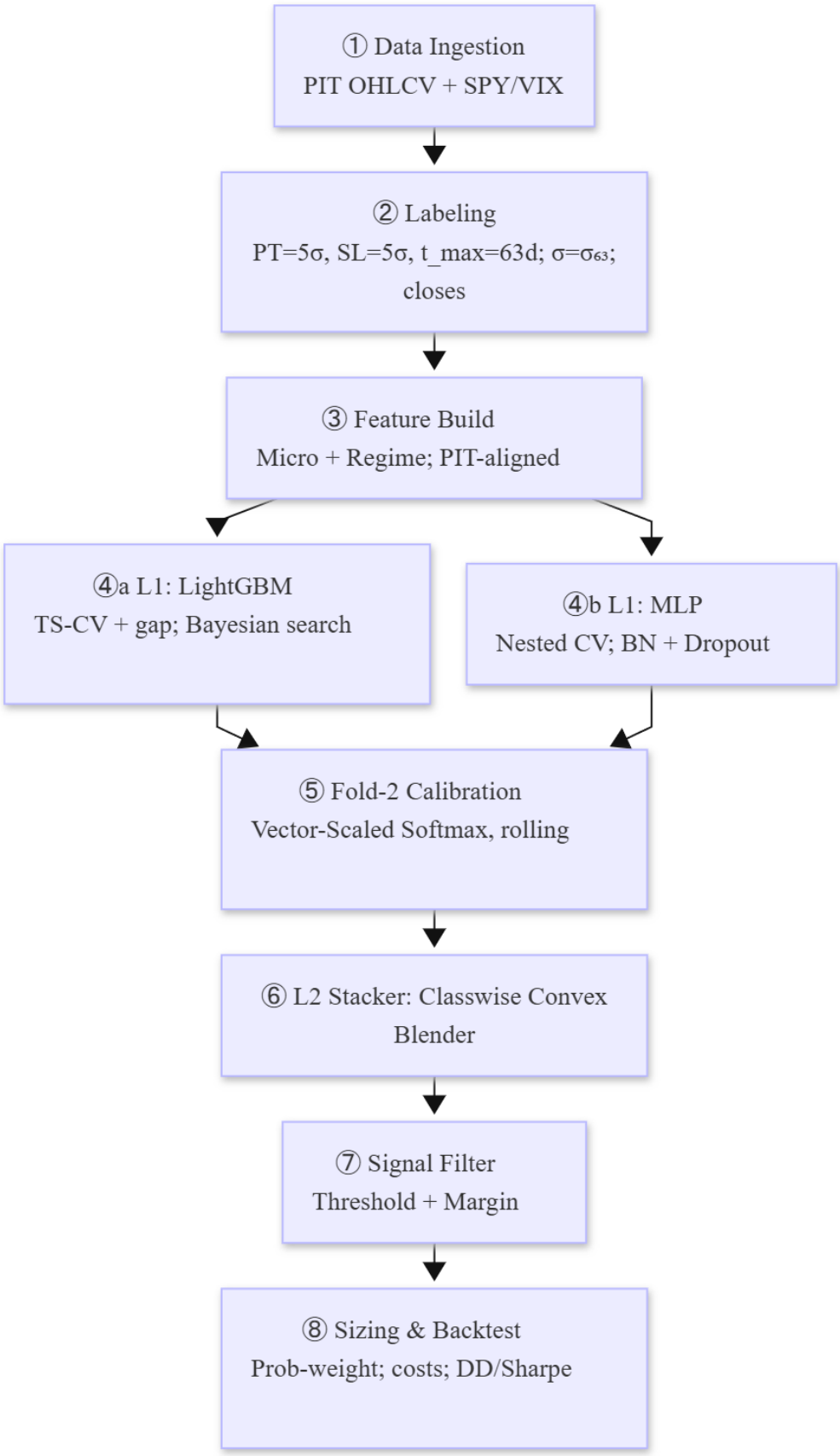

Figure III.2 – Flowchart of the pipeline

VI – Modeling Decisions

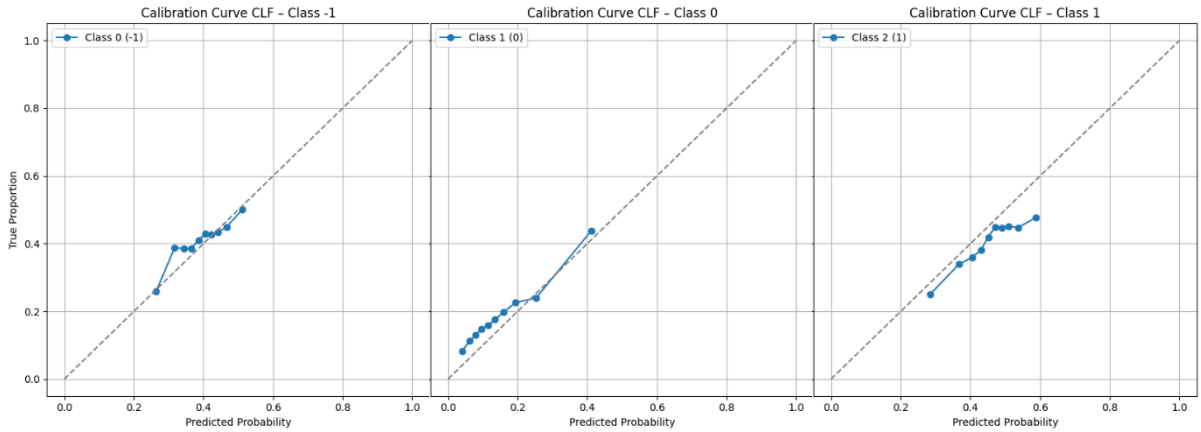

Figure VI.5a — Calibration curves per class [-1,0,+1] of the LightGBM classifier

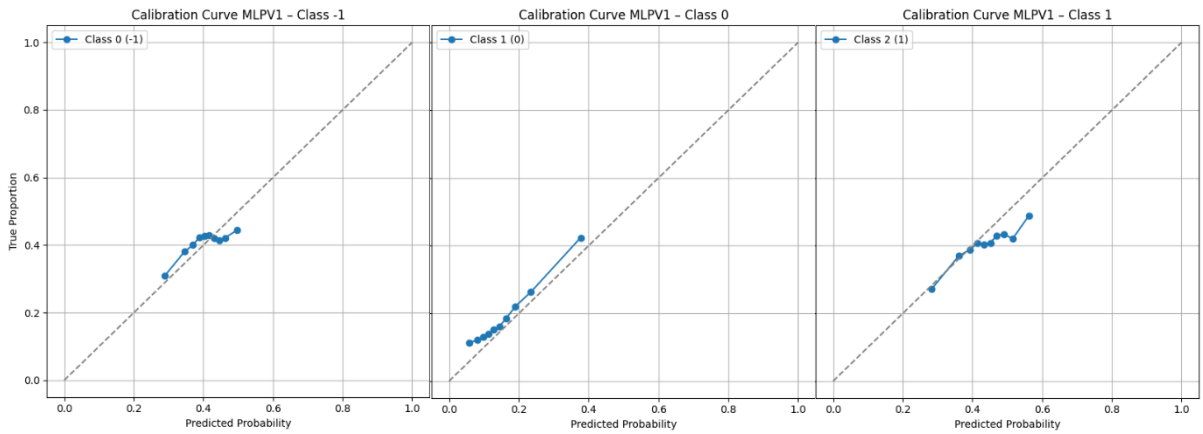

Figure VI.5b — Calibration curves per class [-1,0,+1] of the Multi-Layer Perceptron

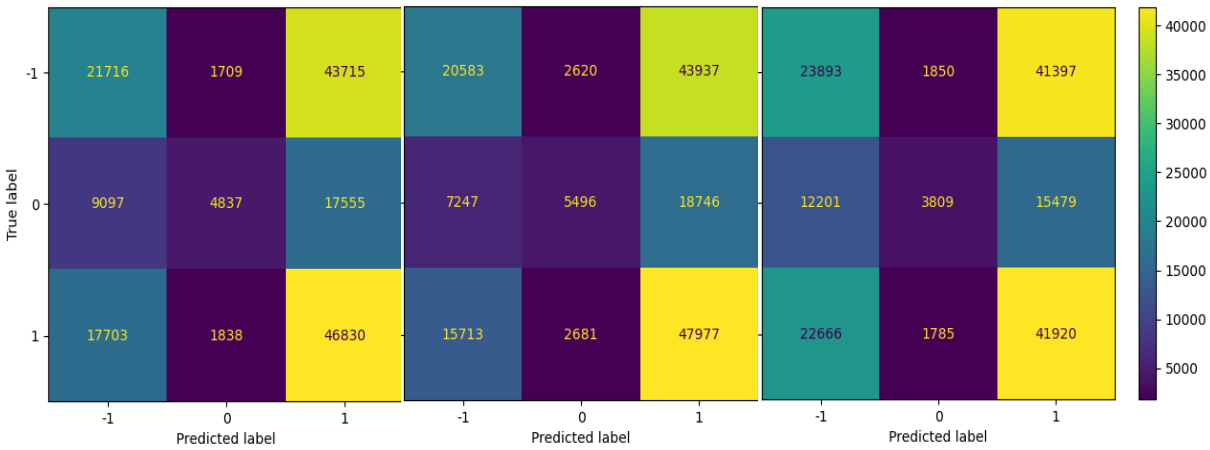

Figure VI.6 — Confusion matrices per model, from left to right: Classwise Blender, LightGBM Classifier, Multi-Layer Perceptron

Table VI.7 — Feature Importance per model

LightGBM Classifier		Multi-Layer Perceptron	
month_of_year_sin	3.52%	yield_curve_slope	8.49%
return_skew_63d	2.81%	volatility_zscore	6.11%
atr	2.71%	market_vol_zscore	4.33%
amihud_illiquidity	2.71%	month_of_year_sin	3.96%
price_max_63d	2.58%	price_percentile	3.84%
cs_z_obv	2.45%	cs_z_rsi	3.75%
cs_z_corr_spy	2.43%	dist_52w	3.44%
time_from_high_252	2.41%	mdd_63	2.97%
adx	2.41%	cs_rank_price_percentil	2.86%
cmf	2.40%	return_skew_63d	2.68%
vol_trend_63	2.34%	cs_rank_mom21d	2.63%
market_returns_zscore	2.33%	beta_63d	2.43%
time_from_low_252	2.31%	mom_1m	2.41%
beta_instability_63	2.31%	time_from_low_252	2.41%
obv	2.28%	cs_z_beta	2.40%
corr_spy_63d	2.28%	corr_spy_63d	2.39%
cs_z_beta	2.25%	10yTbill	2.38%
price_percentile	2.24%	rsi	2.35%
beta_63d	2.10%	cs_z_corr_spy	2.15%
time_from_low_63	2.05%	cs_rank_volume_surge	2.10%
volume_skew_21d	1.98%	es95_63	1.91%
time_from_high_63	1.97%	vol_adj_momentum	1.83%
volume	1.95%	volume_surge	1.80%
mom_1m	1.90%	amihud_illiquidity	1.70%
vov_63	1.88%	return_skew_21d	1.64%
return_skew_21d	1.87%	vix	1.64%
cs_z_volatility	1.86%	time_from_high_252	1.61%
market_vol_zscore	1.85%	time_from_low_63	1.55%
trix	1.77%	momentum_63_21d	1.53%
macd	1.69%	market_returns_zscore	1.41%
mom_12m	1.69%	adx	1.38%
dd_speed_21	1.65%	cs_z_volatility	1.37%
volatility_63d	1.60%	cmf	1.34%
dist_52w	1.54%	mom_persistence_63d	1.30%
vix	1.52%	cs_rank_mom63d	1.23%
yield_curve_slope	1.48%	trix	1.15%
momentum_63_21d	1.46%	dd_speed_21	0.99%
10yTbill	1.46%	volume_skew_21d	0.96%
mdd_63	1.45%	time_from_high_63	0.93%
cs_z_rsi	1.44%	vix_high	0.91%
volatility_zscore	1.43%	momentum_63d	0.75%
cs_rank_mom21d	1.41%	trend_strength_63	0.70%
volatility_21d	1.41%	volatility_63d	0.64%
es95_63	1.38%	obv	0.61%
cs_rank_volume_surge	1.30%	cs_z_obv	0.52%
dsv_63	1.30%	volume	0.51%
trend_strength_63	1.21%	vov_63	0.42%
cs_rank_mom63d	1.19%	volatility_21d	0.35%
rsi	1.11%	vol_trend_63	0.35%
vix_high	1.09%	macd	0.26%
vol_adj_momentum	1.06%	mom_12m	0.21%
cs_rank_price_percentil	1.01%	price_max_63d	0.16%
momentum_63d	1.00%	atr	0.15%
volume_surge	0.97%	dsv_63	0.07%
mom_persistence_63d	0.20%	beta_instability_63	0.02%

Table VI.8 – Classification report of both base models and classwise blender

Class	Precision	Recall	F1-score	Support	Prevalence	ECE	Brier
Classwise Blender							
-1	0.4476	0.3234	0.3755	67140	40.7%	0.0150	0.2385
0	0.5769	0.1536	0.2426	31489	19.1%	0.0350	0.1442
1	0.4332	0.7056	0.5368	66371	40.2%	0.0429	0.2378
Macro avg	0.4859	0.3942	0.3850	165000	—	—	—
Weighted avg	0.4665	0.4447	0.4150	165000	—	—	—

Model Metric	Log Loss	Accuracy	ROC AUC (macro OVR)
Value	1.0211	0.4447	0.5977

LightGBM Classifier							
-1	0.4727	0.3066	0.3719	67140	40.7%	0.0229	0.2382
0	0.5090	0.1745	0.2599	31489	19.1%	0.0391	0.1449
1	0.4336	0.7229	0.5420	66371	40.2%	0.0509	0.2389
Macro avg	0.4718	0.4013	0.3913	165000	—	—	—
Weighted avg	0.4639	0.4488	0.4190	165000	—	—	—

Model Metric	Log Loss	Accuracy	ROC AUC (macro OVR)
Value	1.0255	0.4488	0.5983

Muli-Layer Perceptron							
-1	0.4066	0.3559	0.3796	67140	40.7%	0.0290	0.2408
0	0.5117	0.1210	0.1957	31489	19.1%	0.0326	0.1464
1	0.4243	0.6316	0.5076	66371	40.2%	0.0368	0.2393
Macro avg	0.4475	0.3695	0.3609	165000	—	—	—
Weighted avg	0.4338	0.4220	0.3960	165000	—	—	—

Model Metric	Log Loss	Accuracy	ROC AUC (macro OVR)
Value	1.0309	0.4220	0.5781

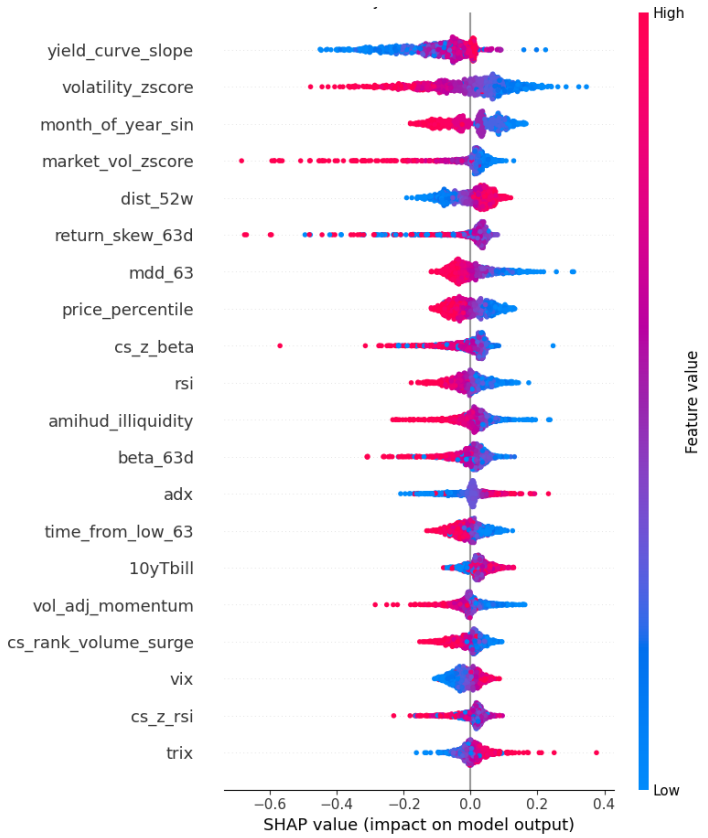

Figure VI.9a – SHAP summary of the Multi-Layer Perceptron for class [+1] (Successful Trades).

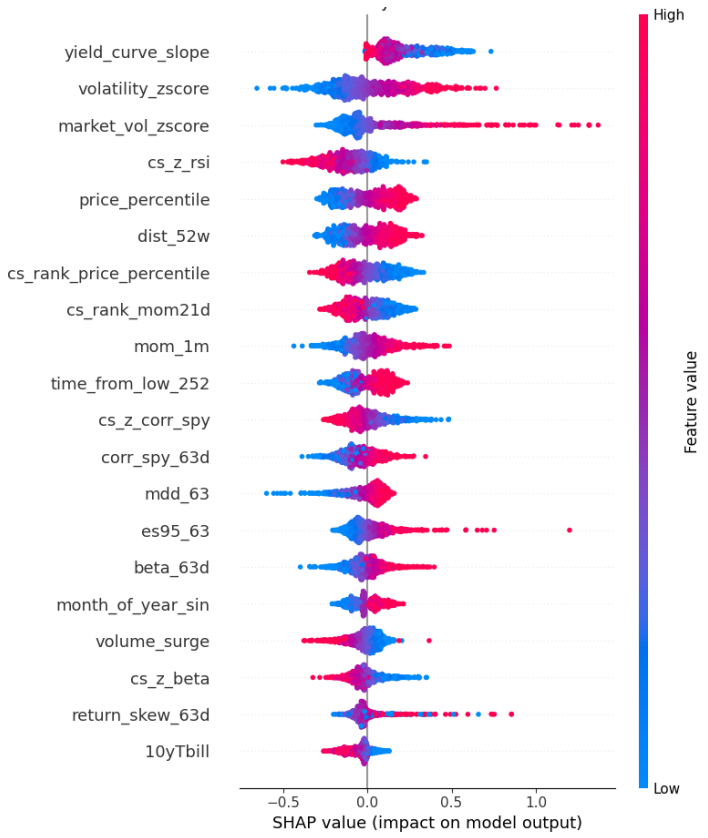

Figure VI.9b – SHAP summary of the Multi-Layer Perceptron for class [0] (Timeout).

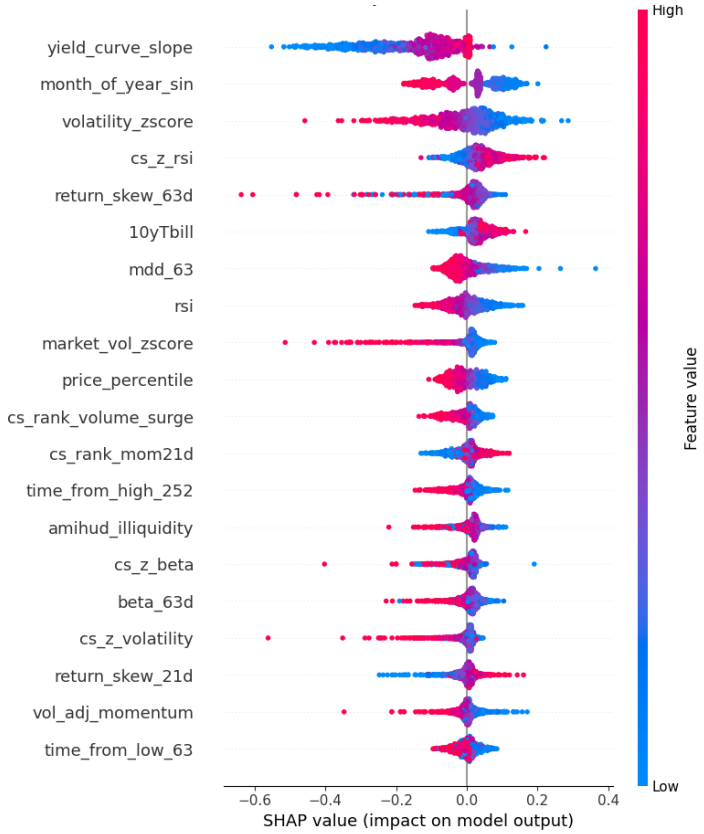

Figure VI.9c – SHAP summary of the Multi-Layer Perceptron for class [-1] (Bad Trades).

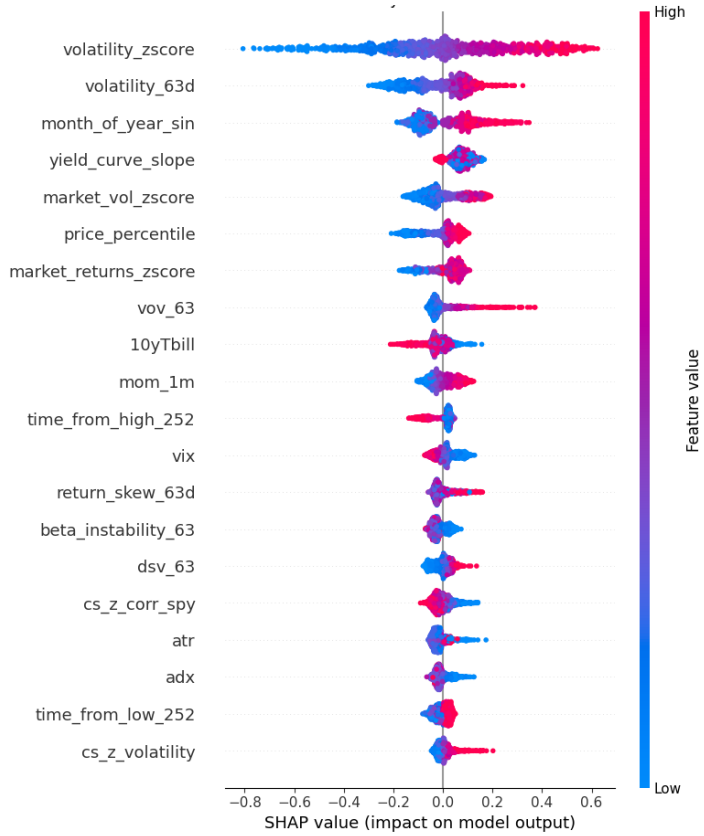

Figure VI.10a – SHAP summary of the LightGBM Classifier for class [0] (Timeout).

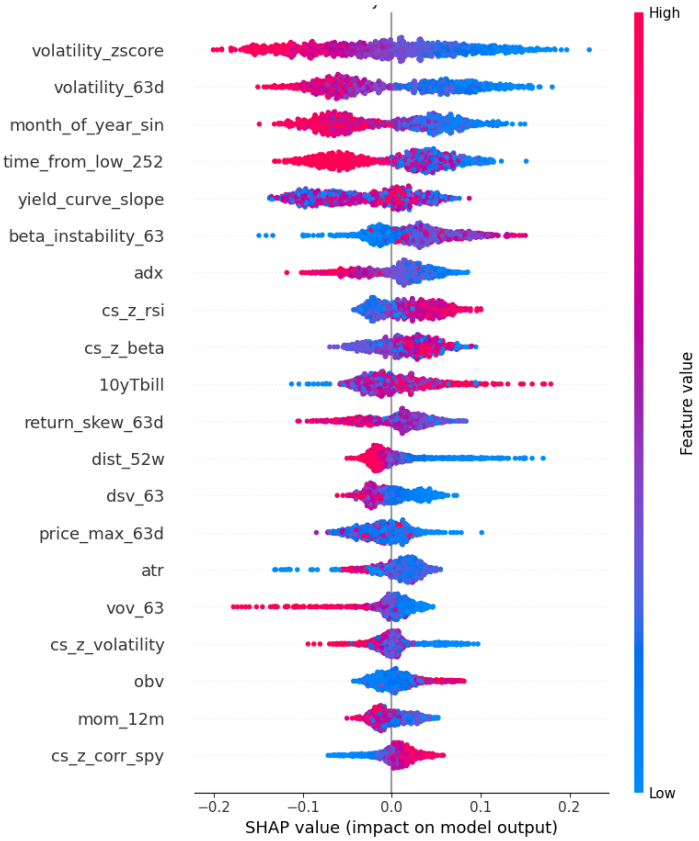

Figure VI.10b – SHAP summary of the LightGBM Classifier for class [0] (Timeout).

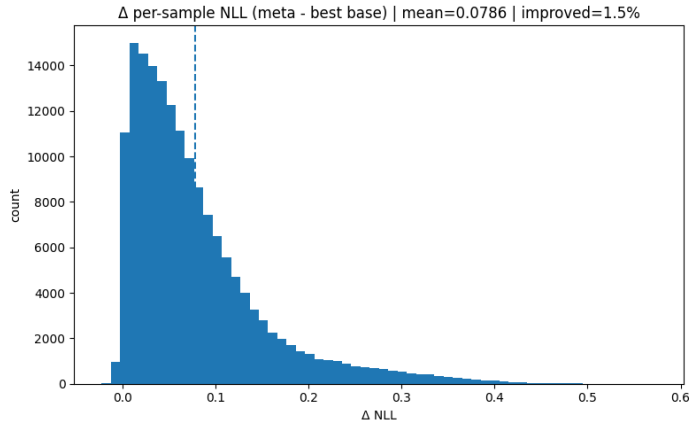

Figure VI.11 – Δ per-sample negative log-likelihood (NLL), meta vs. best base (Fold-3)

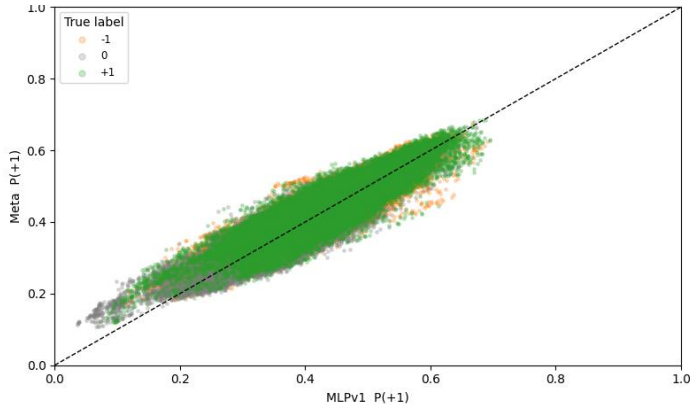

Figure VI.12 – P(+1): MLPv1 vs. Meta (Fold-3)

IX – Key results

Meta-Labeling Strategy

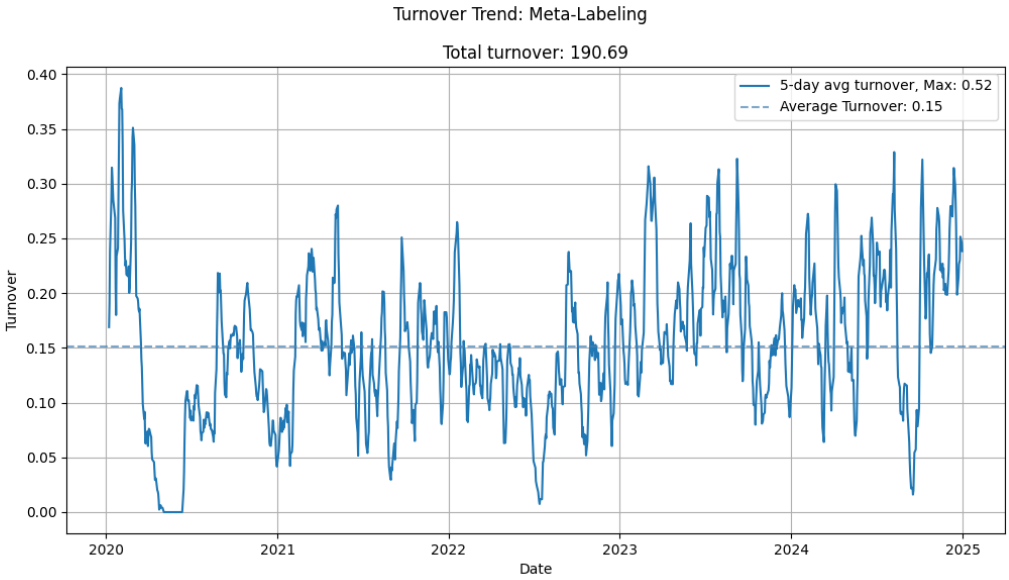

Figure IX.7 – 5-day average turnover trend of the meta-labeling strategy

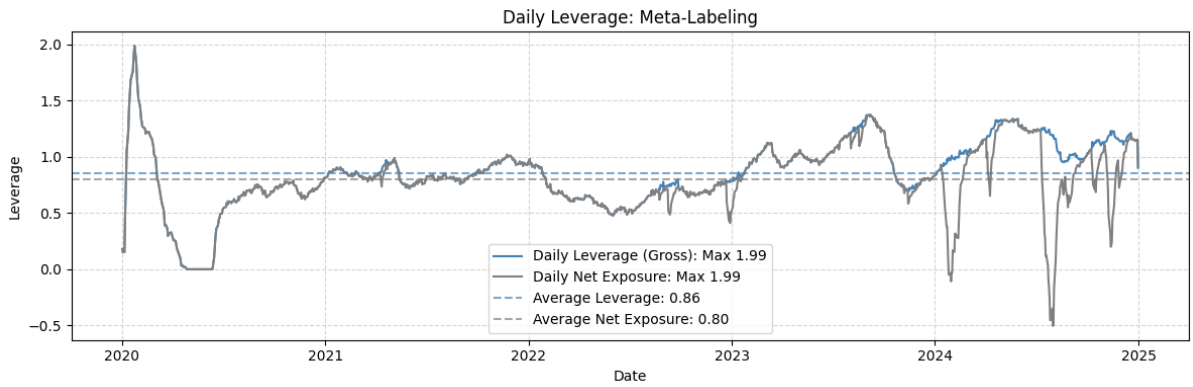

Figure IX.8 – Daily leverage of the meta-labeling strategy

50/50 Blend with SPY

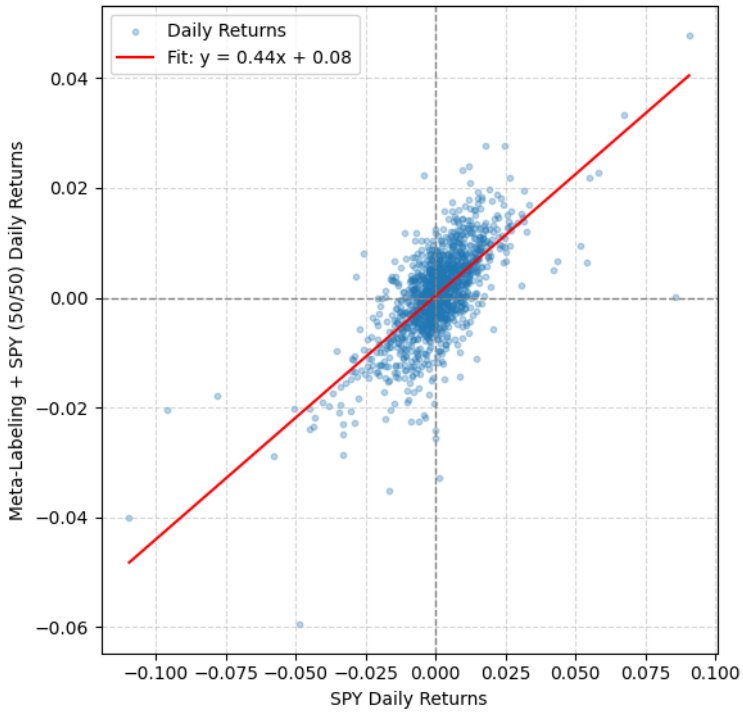

Figure IX.9 — Alpha/Beta scatter of 50/50 blend strategy vs SPY with fit line

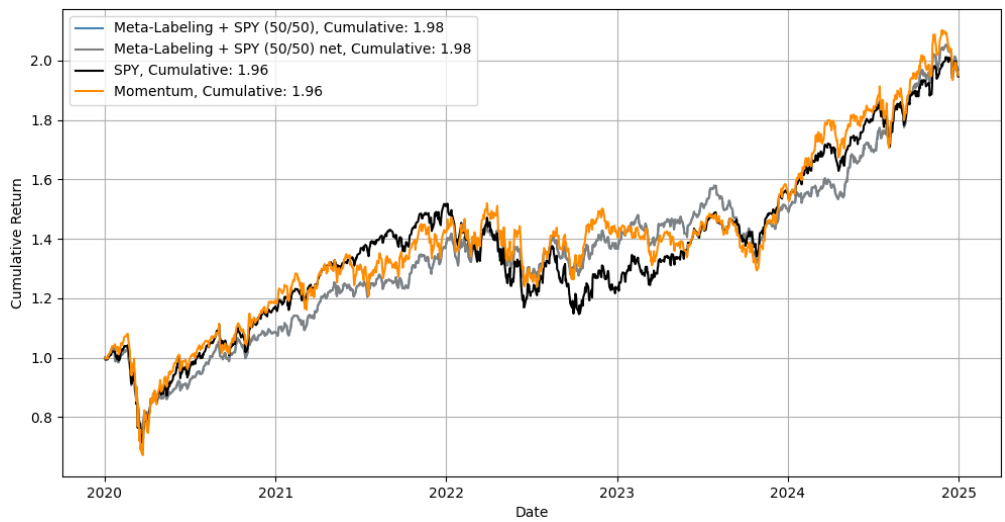

Figure IX.10 — Cumulative returns of 50/50 blend strategy (Meta net, SPY, Momentum)

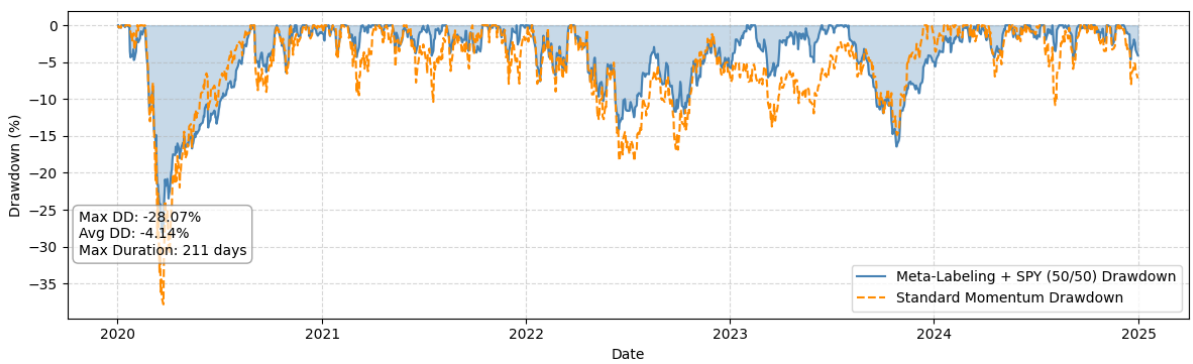

Figure IX.11 — Underwater Drawdown of 50/50 blend strategy vs Momentum

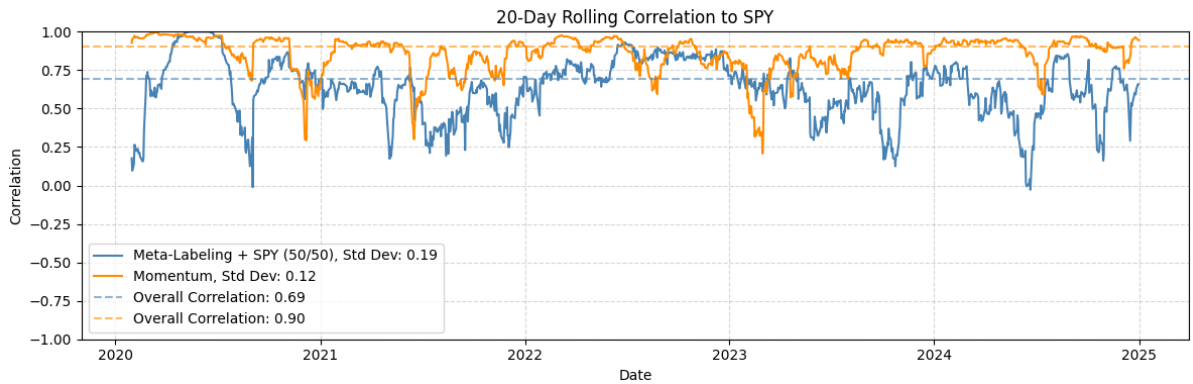

Figure IX.12 — 20-day rolling correlation of 50/50 blend strategy

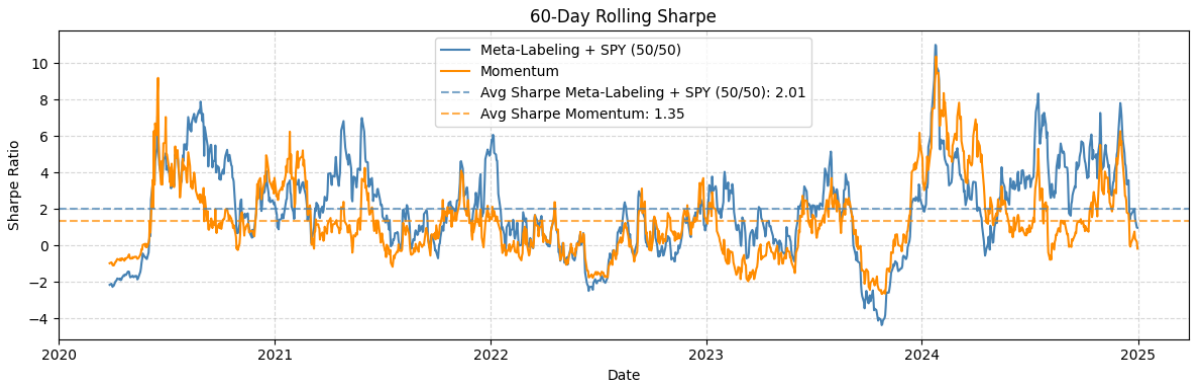

Figure IX.13 — 60-day rolling Sharpe of 50/50 blend strategy

Beta Neutral

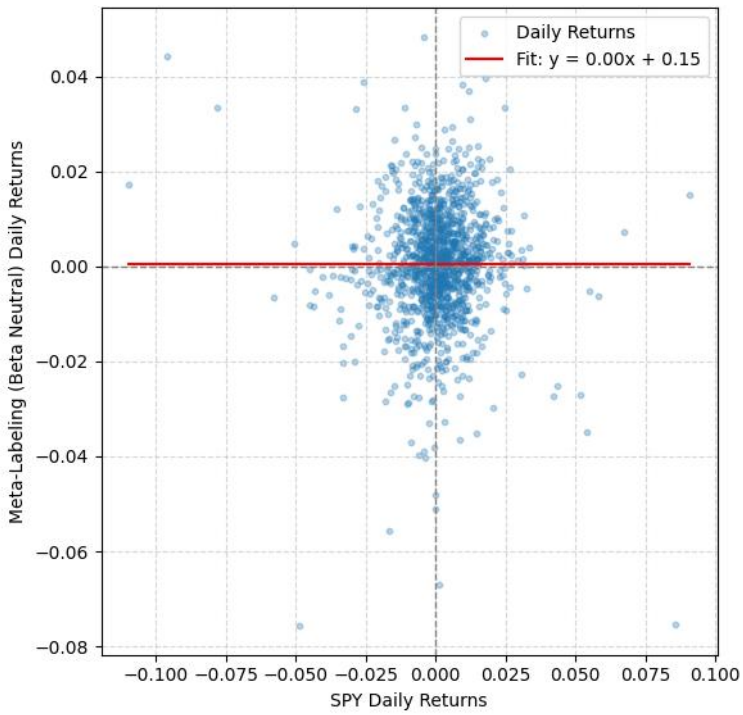

Figure IX.14 — Alpha/Beta scatter of beta neutral strategy vs SPY with fit line

Figure IX.15 — Cumulative returns of beta neutral strategy (Meta net, SPY, Momentum)

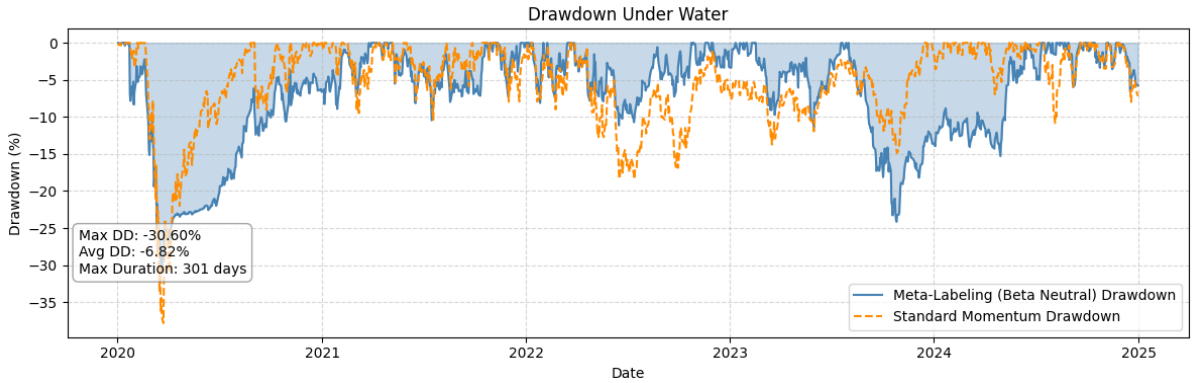

Figure IX.16 — Underwater Drawdown of beta neutral strategy vs Momentum

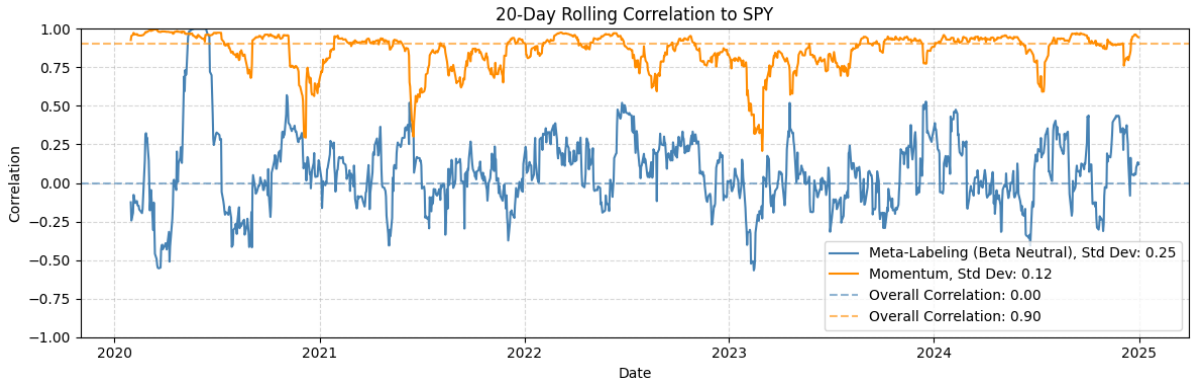

Figure IX.17 — 20-day rolling correlation of beta neutral strategy

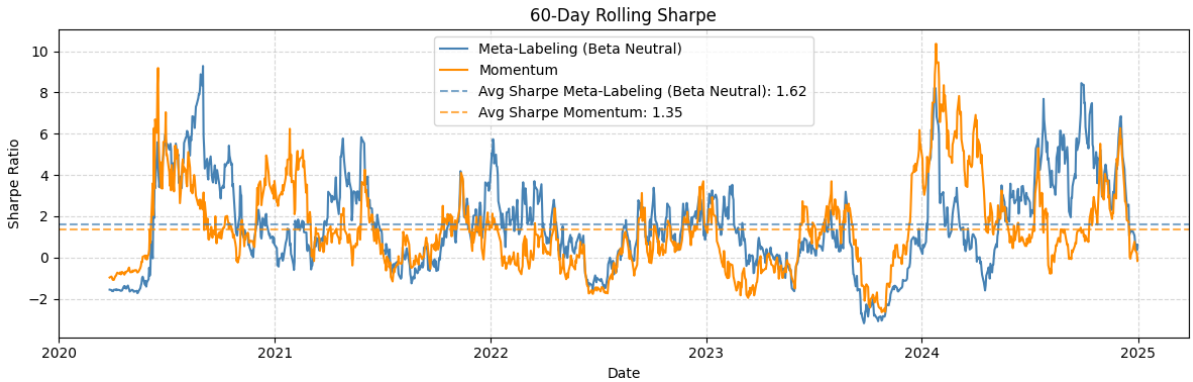

Figure IX.18 — 60-day rolling Sharpe of beta neutral strategy